

AEP: Declared Ambiguity for Non-Idempotent APIs Without Idempotency Keys

Anonymous Author(s)

Abstract—Many enterprise APIs are non-idempotent, accept no idempotency key, and cannot be asked afterwards whether a mutation was applied. A caller that crashes around such a call (an autonomous agent, a workflow engine) has no safe option: retrying risks a second real-world effect nobody observes, not retrying an effect no record accounts for. We argue this is not an engineering-quality problem but a three-way trade whose third corner a system must make reachable: silent failure converted into *declared, durable, bounded ambiguity* an operator can act on.

We present AEP, a fail-closed protocol in which every external side effect is preceded by a durably acknowledged write-ahead intent, every state write is fenced by lock ownership and an expected-version CAS in one atomic script, and every unresolvable outcome escalates rather than guessing. We evaluate it against five baselines under `SIGKILL` faults, across endpoint capabilities differing in what a read-back can settle.

Our central result separates two claims usually sold as one. *Detection* (no undetected duplicate and no lost effect, leaving a residual of declared ambiguity) comes from the pre-dispatch record and a transition table that forbids re-entry into dispatch, not from the durability barrier. *Prevention* is the barrier's, against a narrower fault than it appears: it withholds dispatch when the store dies inside the acknowledgment window, not when storage discards writes while reporting success, which we injected and where it dispatched. Because detection does not depend on the barrier for record-preserving faults, its cost is a deployment choice, not the protocol's price.

Index Terms—Fault tolerance, non-idempotent APIs, idempotence, distributed coordination, write-ahead logging, fault injection, reliability engineering, autonomous agents.



1 INTRODUCTION

An autonomous agent is asked to post a journal entry, issue a refund, or file a regulatory notification. It calls an enterprise API written years before any of this. The call times out, and nothing on either side can say whether the money moved. Autonomous agents are the setting in which this is now most visible, and we use them throughout as the motivating example. The problem itself belongs to the endpoint, and any caller that can fail mid-call inherits it. A workflow engine, a retrying batch job and a human operator with a script are all in the same position.

This is an old problem in a new setting, and the new setting removes the assumption that the old solutions rest on. Durable-execution engines, workflow systems and message brokers achieve their guarantees by requiring that the operation being retried is idempotent, or that the remote side will accept a deduplication key, or that both endpoints can be enrolled in one transaction [1], [2], [3]. Temporal's documentation is explicit about the precondition: "You should always make your business logic Activities idempotent in Temporal. Because Activities may be retried, these functions may be executed more than once" [4]. The legacy endpoints at issue satisfy none of these conditions. They are non-idempotent and they accept no idempotency key. Many of them also cannot be asked afterwards whether a mutation was applied, and that third gap is the one that does the real damage.

1.1 The trilemma

Consider a worker that has sent a mutation and then crashed, and a recovery process that must decide what to do. Exactly three things can happen to the external world, and a system must choose which one it is willing to produce.

- An *undetected duplicate*. The recovery process re-sends. If the first request had in fact been applied, a second real effect now exists and no component of the system knows it. A naive retry produces this outcome, and so, as Section 6 shows, do a lease, a fenced compare-and-swap and a durable event-sourced history, which each fail to prevent it because none of them is a record written before the call.
- A *lost effect*. The recovery process does not re-send and records the attempt as failed. If the first request was applied, a real effect now exists that the system's records deny. An at-most-once configuration produces this outcome. It is quieter than a duplicate and no better: a refund that the ledger says did not happen becomes a reconciliation problem that surfaces in the accounts weeks later.
- A *declared ambiguity*. The system declines to decide, records that it cannot decide, and stops. The effect may or may not exist. What the system guarantees is that a durable record says so, and that no further automated action is taken on that execution.

The first two outcomes are silent and the third is not. Our position is that the third corner is the one a protocol should be engineered to reach. Against an endpoint that cannot be queried, avoiding ambiguity altogether is impossible, so the quantity a system should be judged on is whether its

residual uncertainty is declared and bounded rather than silent and unbounded.

This impossibility is not incidental. Deciding whether a remote effect occurred, when the channel may fail and the remote side may not answer, is the coordinated-attack problem; no protocol resolves it with certainty, and the classical results on consensus under crash faults and on failure detection say why [5], [6]. What a designer can choose is where the uncertainty is allowed to surface. We call our protocol AEP. It records an intent before every external call and fails closed when recovery cannot resolve the outcome. AEP places the uncertainty in a durable state that an operator can see, and in every cell we measured, none of it reached the accounts.

1.2 Contributions

- C1 The declared-ambiguity formulation.** We state the problem as the three-way trade of Section 1.1 rather than as a pursuit of exactly-once. The formulation gives three properties: fenced state, detectable ambiguity, and a fail-closed liveness bound. Each is mapped to the code path that enforces it, and each has its residual window declared rather than assumed away (Sections 3 and 4).
- C2 A protocol and an implementation** in which a durably acknowledged write-ahead intent is a checked precondition of dispatch authority, not a matter of call ordering. Under a trusted-code assumption, the fsync acknowledgment returns an opaque, non-copyable, single-use, scope-bound in-process dispatch guard. The guard is consumed to write a Redis-visible authorization, and the pre-dispatch script refuses to proceed without it (Section 4). This is a control-flow invariant of the supported API. It is not a cryptographic capability, and it is not a boundary against arbitrary code in the same Python process.
- C3 An evaluation under real process kills** across six named crash points, three endpoint reconciliation capabilities and five reported fault regimes, against five baseline designs, one of them in two configurations. The five regimes are every execution crashed, none crashed, Redis hard-killed before the acknowledgment, Redis killed after dispatch, and storage discarding writes while reporting success. (A sixth regime, a 30% crash probability, was collected; its rates are not in these tables and the collection is in the artifact.) AEP records no undetected duplicate and no lost effect in any cell measured, and the baselines without a pre-dispatch record duplicate in most crashed executions. AEP's residual is declared ambiguity, whose rate is a function of what the endpoint can be asked (Section 6).
- C4 A decomposition of the mechanism, by ablation, and the boundary at which it stops.** The write-ahead pattern is normally presented as one mechanism delivering one guarantee, and it is two. The pre-dispatch record together with no re-entry is what makes an outcome detectable, and it does so without the durability barrier: ablating the barrier produces

no observed difference in the crashed-regime detection metrics (Section 6.3.1). The barrier contributes something else, withholding the external call when durability can no longer be confirmed. That contribution is invisible to the crash faults that dominate this literature and large under the fault that targets Redis (Section 6.3.2). Separating the two changes what a deployment must pay for: detection is nearly free, prevention is where the fsync cost lives, and an operator can buy the first without the second (the supplementary material, *The barrier is a deployment choice*).

1.3 What this paper does not claim

We claim no exactly-once external execution, no prevention of a duplicate the provider has already accepted, no high availability, consensus or split-brain immunity, and no durability beyond one local AOF fsync. The system is a single Redis trust domain and the measurements are single-node. These are stated as a table of non-claims in Section 3 with the reason each is out of scope, and revisited in Section 9.

2 MOTIVATING TRACES

None of the four traces below is hypothetical. Each is a run of our harness (Section 6), replayed from its event log and cross-checked against the mock provider's ground-truth ledger, an independently written record of every mutation the provider actually applied. In each, the workload is the same: acquire work, call one non-idempotent endpoint, record the outcome. It is a scripted caller, not an agent. The traces show what the endpoint does to a caller that crashes, which is the property an agent deployment would inherit.

2.1 Trace 1: the duplicate nobody sees (B0, naive retry)

A worker sends the mutation. The provider applies it. The worker is SIGKILLED after the response is on the wire but before the outcome is recorded (crash point `after_response_before_resolution`). A supervisor observes an execution with no terminal record and runs the step again. The provider applies the mutation a second time.

The ground-truth ledger now holds two applied mutations for one logical step. The system's own records hold one successful execution. Nothing in the system is in a state that a monitor could alert on: there is no error, no retry counter above its threshold, and no ambiguous record. The execution is green.

This is not a strawman configuration. It is what a retry-on-timeout loop does, and it is the behavior we measure for B0. Across the crashed cells B0 ends this way in 0.74–0.78 of executions depending on the endpoint, and so do B1 and B2 (Table 7).

2.2 Trace 2: a lease and a fenced write do not help

We run the same crash against B1 (a real Redis lease) and B2 (a lease plus an exact expected-version compare-and-swap on the state document). Both are the patterns an experienced engineer reaches for, and both are doing their job: no stale writer corrupts the state document, and no two workers hold the lock at once.

Neither prevents the second effect, because the mechanisms that protect the database do not protect the world. The lease governs who may write state, and the CAS governs which version of state may be replaced; the external call is neither. When the supervisor re-runs the step it holds a valid, freshly acquired lease, and the correctly versioned record it writes is a record of a duplicate. Section 6 shows B1 and B2 duplicating at substantially the same rate as B0.

2.3 Trace 3: a durable log is necessary and not sufficient (B4)

B4 is an event-sourced durable-execution engine of the kind these systems are built on: an append-only history in Redis, each append acknowledged durable with the same WAITAOF barrier AEP uses, replay after a crash, and completed activities memoized so they are not re-run. It is not ablated on durability, and its history is on disk before the call.

It duplicates anyway. The history records that the activity was scheduled. It does not record whether the provider applied the effect, and cannot, because that fact was never available to the engine. On replay the engine sees a scheduled activity with no completion and schedules it again, following the vendor’s documented default, under which Activity Maximum Attempts is unlimited [7].

The mitigation the vendor recommends is to make the activity idempotent [4]. That is exactly the precondition this paper’s premise removes.

2.4 Trace 3b: the at-most-once configuration loses the effect

Setting Maximum Attempts to 1 is documented and standard: “Setting the value to 1 means a single execution attempt and no retries” [7]. We run it as B4b. It cannot produce a caller-caused duplicate, and it does not.

What it produces instead is the other silent failure. Where the provider applied the mutation before the worker died, B4b’s history records an activity that timed out, the workflow fails, and the effect exists while the records deny it. That record is an ordinary failure and not a declared ambiguity. B4b has no outcome class in which the engine can say that it does not know, so nothing escalates and no operator is told that a real-world effect may be unaccounted for.

2.5 What the traces establish

Table 1 is the shape of the argument. At-most-once dispatch is available off the shelf: B4b buys it with one configuration setting, and so does AEP’s ablation B3. What is not available off the shelf is knowing which of duplicate-or-loss occurred. The fact a system would need for that, whether the provider applied the effect, is not in any log it can write and cannot be put there by any amount of logging. It has to be obtained from the endpoint, and Section 6.2 shows the price of that: the achievable outcome is bounded by what the endpoint can be asked.

TABLE 1

The trilemma, as the systems under comparison instantiate it. A durable write-ahead record is necessary (Traces 1, 2) and not sufficient (Trace 3); what distinguishes AEP is an outcome class in which it can decline to decide. B3, which is AEP with the durability barrier removed and nothing else, reaches the same corner, so the barrier is not what buys declared ambiguity (Section 6.3.1 measures the ablation), and Section 6.3.2 isolates what it does buy.

	<i>undetected duplicate</i>	<i>lost effect</i>	<i>declared ambiguity</i>
B0 naive retry	yes	—	no
B1 lease-only	yes	—	no
B2 CAS-only	yes	—	no
B4 durable, max attempts ∞	yes	—	no
B4b durable, max attempts = 1	no	yes	no
B3 AEP minus the barrier	no	no	yes, bounded yes,
AEP-full	no	no	bounded

3 SYSTEM AND FAILURE MODEL

This section condenses a formal model held in the artifact, which states every claim below with a citation to the enforcing code by file and line and is machine-checked in CI: a script fails the build if a cited path stops existing or a cited line falls outside its file. That check proves the citation ranges are valid, not that the semantics are right. The semantic evidence is the per-property regression tests named in Section 4.

3.1 Principals and store

A *runner* performs at most one external mutation per invocation. A *recovery service* is read-only with respect to the external system and writes only coordination state. A *connector* is the only component that touches the external API. The store is a single self-hosted Redis 7.2 instance with the append-only file enabled and RDB snapshots disabled. There is no replica, no Sentinel and no Cluster.

Redis executes a Lua script without interleaving, so a script is a single serialization point. This is the only atomicity primitive the protocol has and every invariant below is expressed inside one.

3.2 Clocks

There is no synchrony assumption beyond lease TTLs. All ordering decisions between principals are taken against the Redis server clock; no cross-worker clock comparison occurs. Process-local monotonic time is used only for heartbeat intervals and watchdogs, never for an authoritative decision. The lease budget $T_{\text{client}} \leq T_{\text{lock}} - B$ with $B \geq 15$ s is enforced at configuration time. It is a timing budget rather than a guarantee: a scheduling gap remains between a successful preflight and transmission, and we declare it as such.

3.3 The external endpoint

The endpoint is non-idempotent and non-cooperative: it need not accept an idempotency key, and its response may

TABLE 2

Endpoint reconciliation capability. The right-hand column is the ceiling on what any protocol can conclude, and Section 6.2 shows AEP's declared-ambiguity rate tracking it.

Capability	Conclusions the endpoint can support
AUTHORITATIVE READBACK	applied, not-applied, unknown, conflict. Absence <i>proves</i> the mutation did not occur.
POSITIVE-ONLY READBACK	applied, unknown, conflict. Presence can be confirmed; absence proves nothing.
NO-READBACK	none. The endpoint cannot be queried at all.

be absent, late, or self-contradictory. Two things about it are modeled explicitly, and the second is the axis the evaluation turns on.

Evidence at mutation time: Ambiguity is the default here, not a special case. The runner accepts only policy-declared success and failure values, required non-empty and disjoint; any other value, and any exception whatever, is coerced to ambiguity.

Reconciliation capability: What the endpoint can be asked after the fact is a declared, typed property with three values (Table 2). The declaration fails closed: a missing, null or unrecognized capability is treated as the weakest, not as the strongest. Classification is total. In particular, “the endpoint reports the mutation absent” combined with a positive-only capability is a *named contract violation* that resolves to ambiguity, not a fall-through to “it did not happen”.

3.4 Failure model

The model is crash-and-delay, not Byzantine. Redis is trusted to execute scripts atomically and honor expiry; the provider is trusted only insofar as its declared evidence is truthful.

- F1 Worker crash at any instruction boundary.** Six named crash points around the external call (Table 3). In the evaluation these are OS-level `SIGKILL` of a separate worker process, not in-process exceptions.
- F2 Partition between a worker and Redis.** Any Redis command may raise; every raise forbids progress rather than permitting a guess. A partition outlasting the lease is observationally equal to F5.
- F3 Redis restart with AOF replay.** Up to roughly one second of acknowledged-but-unsynced writes may be absent under `appendfsync everysec` [8]. Section 6.3.3 reports two probes that sharpen this considerably. The set of faults that can actually realize that loss is narrower than the documentation's wording suggests, and a process kill is not among them; a host-level write loss is, and realizes the loss every time.
- F4 Delayed or duplicated external responses.** A late effect cannot be recalled; recovery waits out a reconciliation delay before drawing any conclusion, and never re-dispatches.

TABLE 3

The six crash points, and what exists in the world at each *when the kill is delivered at the position named*. The right-hand column is why the declared-ambiguity rate in Section 6.2 varies so sharply across them.

This table describes the intended position. For the five baseline systems the kill at `after_barrier_before_dispatch` was delivered later than the position named here, and Section 6.1 states what was delivered instead.

Crash point	State when the worker dies
<code>before_intent_write</code>	no record, no effect
<code>after_intent_before_barrier</code>	record in memory, maybe not on disk; no effect
<code>after_barrier_before_dispatch</code>	record durable; no effect
<code>mid_dispatch</code>	record durable; effect likely
<code>after_response_before_resolution</code>	record durable; effect known to the dead process only
<code>after_resolution_before_barrier</code>	outcome written, not yet acknowledged

- F5 Worker pause past lease expiry** (GC or VM stall), handled by an atomic pre-dispatch preflight, cancellation of the protected task on ownership loss, and CAS rejection of the late write.

Out of model: Byzantine Redis; a provider whose declared evidence contradicts its own effects; loss of the Redis host once the `fsync` acknowledgment has been returned; adversarial writers with direct socket access; compromise of vault keys.

3.5 Non-claims

We state these because reviewers are right to look for them, and because several are structural rather than incidental.

4 THE AEP PROTOCOL

4.1 Ordering

The runner's sequence, with the property each step exists to serve:

- 1) Acquire the lease with jittered retry.
- 2) Require that execution state already exists.
- 3) Prepare, vault and read back the request *before any intent exists*, so that the intent, once written, names a request that is already durable.
- 4) Write the intent `NONE`→`ABOUT-TO-FIRE` by fenced CAS, **and** run the durability barrier on the *same pinned connection*.
- 5) Convert the barrier's acknowledgment into a Redis-visible dispatch authorization.
- 6) Run the atomic pre-dispatch preflight, which re-checks that authorization.
- 7) Mint a single-use dispatch capability.
- 8) Only then call the connector.

Two of these steps can refuse, and both refusals happen before any provider bytes exist, so both may soundly record a confirmed failure. If the barrier fails at step 4, the runner makes one fenced attempt to record `FAILED-CONFIRMED` with evidence `LOCAL_NO_DISPATCH`. If the preflight rejects

TABLE 4
Non-claims, with the reason each is out of scope.

Non-claim	Why
Exactly-once external execution	Redis and a legacy provider share no transaction coordinator. The protocol converts this into declared ambiguity.
Duplicate <i>prevention</i>	AEP prevents <i>automatic</i> duplicates. An effect the provider already accepted cannot be recalled.
HA, consensus, split-brain immunity	Single instance; the lock is a lease [9]. This is also why the AOF-rewind residual below has no local fix.
A hardened trust domain	No Redis ACL; any principal reaching the socket can write the state key directly.
Durability beyond one local AOF fsync	The barrier requires one <i>local</i> fsync [10]; it is explicitly not a substitute for replication.
Operator alerting	Not implemented. “Escalates” means <i>reaches a terminal automated state and pauses the execution</i> , nothing more.
A verified implementation	The properties are model-checked (Section 4.6); the Python is not. Refinement from the specification to the code is neither proven nor claimed.

at step 6 (a lost lease, a moved version, an absent authorization), the runner does the same, with a reason naming the preflight rather than the barrier, provided it still owns the lease. Either way, if ownership has already been lost the runner records nothing and leaves the intent for recovery. Together these are the paths that produce the measured result of Section 6.3.2. The outcome classifier keys on the resulting status rather than on which of the two refused, so they are counted alike.

4.2 P1: Fenced state

Property 1 (Fenced state). No committed state write can be superseded and then resurrected by a stale writer. Every write to the state key is admitted only if, in one atomic Lua invocation, (a) the caller’s ownership token equals the live value of the lock key, and (b) the stored version equals the caller’s expected version and the candidate’s version equals expected + 1.

Both conjuncts are necessary and the failure of either is a known pattern. Version-only CAS admits a writer that lost its lease but holds the current version; token-only checking admits a writer rebasing on a stale read. The pairing is the fencing-token argument made against lease-only locking [11], with the token check and the write placed inside one script so they cannot interleave. The same script additionally freezes, inside the atomic region, the execution identity, the schema version, every envelope field outside a declared mutable set, every other ledger entry, and the intent’s immutable fields. Transitions are append-only with a prefix check, and ledger entries can never be deleted.

Declared residuals: A plain SET from any client with socket access is unconstrained (there is no ACL). Expiry, not deletion, ends the guarantee: after the retention floor elapses the record is gone and any writer may create version 1 again. An AOF rewind can also un-fence, and that is confirmed by a probe rather than argued. P1 is a statement about one monotonic Redis timeline, and lock keys are by design not barriered, so a lease whose release was lost can reappear alongside a rewound version and a previously fenced writer then satisfies both conjuncts. There is no local fix; the fix is consensus, which is a non-claim (Table 4).

4.3 P2: Detectable ambiguity

Property 2 (Detectable ambiguity). If a *durable* ABOUT-TO-FIRE intent exists, then under F1–F5 it eventually reaches exactly one *terminal automated* state (FIRED-CONFIRMED, FAILED-CONFIRMED or PERMANENTLY-AMBIGUOUS) and no automated transition moves it afterwards; it is never silently re-dispatched and never silently dropped. This holds provided the recovery loop runs, survives its own pass, can eventually obtain the lease, and is not made eligible before the dispatch window closes.

Terminal automated is exact rather than cautious. The only edges out of PERMANENTLY-AMBIGUOUS are the two operator transitions of Section 4.4, so an intent that has been escalated can still be resolved later by a person who holds the lease and the exact version and asserts something the protocol cannot check.

The design question P2 raises is what makes “barrier before dispatch” more than a convention. In this protocol it is a checked precondition, in four links:

- 1) Through the supported API, the barrier issues a `DurabilityAck` only when `WAITAOF` returned at least one local fsync.
- 2) Ordinary callers cannot construct, subclass or copy `DurabilityAck`; the in-process registry detects reuse and binds the guard to `execution:intent:prepared_version`, so a guard for one attempt cannot authorize another through the supported path.
- 3) Authorizing dispatch *consumes* the ack, then runs a script that re-checks lease, version, status and binding digest and writes an authorization key.
- 4) The preflight script requires that exact value; an absent authorization defaults to a value that cannot match, so it fails closed.

The irreducible gap, stated: Redis exposes no way for a Lua script to verify that a previous `WAITAOF` succeeded. A complete in-store proof is therefore impossible. Under trusted-code assumptions, an authorization records that the supported path received and consumed its ordinary barrier guard; it is not an independent proof that Redis fsynced. The HMAC and registry are mutation/reuse checks, not a cryptographic security boundary. Python underscore naming and closure placement do not prevent arbitrary same-process code from calling the module-internal issuer without running the barrier. A principal with direct Redis access can also write the authorization key, and a caller inside the process can compose the binding service with a connector

TABLE 5

Recovery classification. The two rows in bold are where declared ambiguity is produced by design rather than by failure, and they are what Section 6.2 measures.

Observation	Outcome
capability undeclared / unrecognized	ambiguous , no query
NO-READBACK applied (either querying class)	ambiguous , no query
not-applied + AUTHORITATIVE	confirmed
not-applied + POSITIVE-ONLY	refuted
conflict	ambiguous (named violation)
unknown, budget remaining	ambiguous
unknown, budget exhausted	stay unresolved, backoff
	ambiguous

directly, bypassing the runner. A stronger attacker model requires a separately enforced process or service boundary, which this implementation does not have.

A fifth link, and it is a clock: The four links above run from the barrier to the dispatch. They are silent about the other end, and P2 needs it: recovery must not classify an intent whose worker may still be about to transmit. Nothing structural prevents that, because both principals are acting on the same fenced state and a lease can expire at any moment. What prevents it is a delay. An intent becomes eligible for recovery only after $T_{\text{client}} + B + T_{\text{settle}}$ has elapsed since it was prepared, which is sized to cover the whole dispatch window. Remove that delay and the protocol loses an effect: recovery reads back *not applied*, truthfully, because the request has not been sent yet, and records a confirmed failure that the dispatch then contradicts. Section 4.6 produces this in ten steps. The delay is therefore a precondition of P2's soundness rather than a courtesy about late responses. It is also a real-time argument, which is why the residual named in Section 3.2 is the gap it cannot close.

Recovery classification: For an eligible intent, under a freshly acquired lease and after re-validating status and version, the mapping from observation to outcome is total (Table 5). Recovery has no reference to the mutation call at all and can only read back. The transition table has no edge back to ABOUT-TO-FIRE, checked in Python and re-checked inside the atomic script.

False-positive ambiguity is the price, and it is the right direction: A crash after a durable intent but before transmission yields an intent that recovery must treat as ambiguous even though no effect exists. This inflates the declared-ambiguity rate; it can never inflate the undetected-duplicate rate. Section 6.2 shows this asymmetry directly in the data: ambiguity is highest at exactly the crash point where no effect can possibly exist.

4.4 P3: Fail-closed liveness bound

Property 3 (Fail-closed bound). Automated reconciliation of one intent terminates within a configurable attempt budget and duration budget, after which the intent is placed in a terminal automated state and the execution is withdrawn from normal scheduling.

Defaults are 8 attempts and 24 hours, with full-jitter backoff. Attempt counting is monotone and re-checked in-

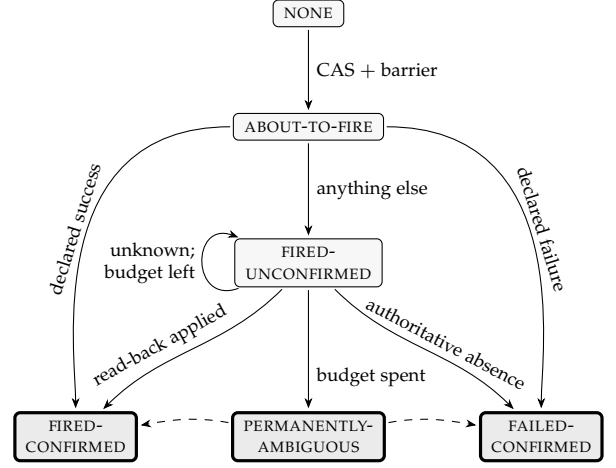


Fig. 1. The intent state machine, generated from the transition table in the implementation. Solid edges are automated; the dashed edges from PERMANENTLY-AMBIGUOUS are the operator-only transitions. Every state with a thick border is terminal for automated reconciliation. There is no edge back into ABOUT-TO-FIRE. That absence is what “never silently re-dispatched” means, and it is enforced inside the atomic script rather than by the caller.

side the atomic script: an unresolved-to-unresolved edge must increment the counter by exactly one. On exhaustion the intent becomes PERMANENTLY-AMBIGUOUS, the execution is paused, no new intent may be created for it, and the only exit is an operator transition that itself requires the lease and the exact version.

Declared residuals: The bound is consulted only on the *unknown* branch. Every other observation resolves immediately, so this is not a gap in practice, but it is not a global watchdog either. Lease contention does not consume budget. There is no escalation *mechanism*: “ejects to operator escalation” means reaching a terminal state and pausing, and nothing alerts. Escalated records also expire: a retention floor of 31 days is enforced for any ledger containing an unresolved or ambiguous record, but a floor is not permanence, so after it elapses the escalation evidence is deleted without an operator having acted. Both residuals are confirmed by executable probes in the artifact, not asserted.

4.5 The state machine

Figure 1 is the transition set exported from the implementation, not a drawing of it, so the presence or absence of an edge is a fact about the code. Three states are terminal for automated reconciliation: FIRED-CONFIRMED, FAILED-CONFIRMED and PERMANENTLY-AMBIGUOUS, which is the declared-ambiguity corner. Nothing returns to ABOUT-TO-FIRE: once an intent leaves the dispatch window it cannot re-enter it, and that absence is P2’s “never silently re-dispatched”. The dashed edges are the operator transitions of Section 4.4, and they are the only way out of that corner.

4.6 Model checking

We specify the protocol in TLA+ and check it with TLC. The specification is transcribed from the implementation, meaning the Lua guards, the exhaustive transition table, the creation fence and the recovery classification, and not

TABLE 6

Seven of the nine configurations that must fail. Each removes one stated assumption; *steps* is the length of the counterexample TLC returns. Two further configurations, not shown, are non-vacuity witnesses: P2's liveness property is an implication, and they establish that its antecedent is reachable rather than letting it pass while meaning nothing.

Assumption removed	Property that breaks	Steps
A single Redis timeline	P1 monotonicity	4
A truthful <i>f_{sync}</i>	no lost effect	8
The barrier, plus a restart	no lost effect	8
A truthful endpoint	no lost effect	9
<i>same, duplicate horn</i>	no undetected duplicate	16
The reconciliation delay	no lost effect	10
Operator authentication	no lost effect	12

from this section. A gate re-derives the transition table and the status alphabet from the source and fails if the two disagree, because a model checker asked about the wrong state machine does not complain: it reports success.

The endpoint is modeled as Section 3.3 describes it, and this is the modeling decision the result rests on. It is never given an intent identifier, so it cannot deduplicate and a second dispatch yields a second effect; whether an effect is applied is chosen adversarially rather than derived from anything the caller did; and ambiguous evidence is reachable on every path, so no caller can ever separate *no effect* from *effect, answer lost*. A god-view variable records what really happened at the endpoint, and no protocol action may read it, which is enforced textually across the specification's protocol definitions, because a guard that could see it would verify a protocol strictly stronger than the implementation while every property still came back green.

Checked: P1 as version monotonicity and the absence of status regression; P2 as legal-edges-only, no re-entry into ABOUT-TO-FIRE, at most one unresolved intent, and eventual resolution under fairness; P3 as the bounded attempt budget; and the trilemma as two invariants: no undetected duplicate, and no lost effect. Declared ambiguity is not a violation, and that asymmetry is Section 1.1 stated formally.

Not checked, and the exclusions are the interesting half. Every cryptographic construction (the request binding, the vault, the digest) is out of the model rather than modeled badly, because an uninterpreted function standing in for a hash proves nothing about the real one and assuming it unforgeable would assume exactly what is in question. The in-process acknowledgment chain of the four links above is *assumed*, not checked: what the model checks is the part that is atomic and Redis-visible, the authorization key and the preflight that requires it. Real time is not modeled. Where that cuts in the protocol's favor it is neutralized, since a lease may expire at any instant, which is more adversarial than the budget; where it cuts against, it is the reconciliation delay named above. Liveness is checked within the version bound, so it establishes that nothing gets stuck for a reason internal to the protocol, not termination in an unbounded run.

Nine configurations exist to fail: Six configurations check that the properties hold. The other nine each remove

one stated assumption and must produce the declared counterexample; a configuration that passes when it should fail turns the build red, which is what keeps this from being a check that cannot fail. Table 6 is where the section's content actually is. Two rows are worth reading twice. The AOF-rewind residual of Section 4.2, previously argued and then confirmed by a probe, falls out in four steps once the single-timeline assumption is dropped. The truthful-*f_{sync}* row is the write-loss result of Section 6.3.3 in the model: the barrier is issued, it reports success, the write is discarded, and an effect ends up with no record at all.

The barrier is a query about durability, not a guarantee: The ablated barrier of B3 returns success without asking anything. The real barrier under write loss returns success because the server answers that it *f_{sync}*ed. These are the same state in the model, and the specification expresses them with one expression, which is the sharpest available statement of what the mechanism is worth: **the barrier is not a durability guarantee but a question about durability, and it inherits the truthfulness of whatever answers it.** The two are not quite the same system. The ablated barrier cannot report failure, where the real one still fails when the server does not answer, so write loss removes the barrier's ability to inform while leaving its ability to fail.

What this establishes, and what it does not: TLC explored every reachable state under these bounds and found no violation. That is a fact about a specification of a few hundred lines, and four things bound what it transfers. The model is *not* the implementation: nothing connects them but a hand transcription and one gate over the transition table and status alphabet, so refinement is neither proven nor claimed, and a defect in the Lua's JSON handling or arithmetic is invisible here. Redis is *assumed*: script atomicity, expiry and WAITAOF semantics are premises, and Section 6.3.3 is the standing demonstration that one of them can be false in a way nothing detects. The scope is *small*: two workers, two intents, five versions, one step, so a defect requiring a third worker or a second step would not appear. And the properties listed above as not checked remain unchecked, most of the implementation's security argument among them. What model checking changes is narrower than it looks and still worth having: the composition of the fences, the state machine and the classification table was previously an argument over code paths and is now a checked one, together with the price of each assumption it rests on in Table 6.

5 IMPLEMENTATION

AEP is 6 862 lines of Python 3.13 against Redis 7.2.5, pinned by image digest. The evaluation harness, mock provider and six baseline systems are a further 30 397 lines.

What the protocol rests on: Three Lua scripts carry the intent path, each a single serialization point: the intent compare-and-swap that holds the whole transition table, the pre-dispatch preflight, and the dispatch-authorization writer. Three further scripts implement the lock's release and renewal and the generic document compare-and-swap; they are not on the dispatch path and none of the properties of Section 4 rests on them. A Lua re-implementation of the strict UTF-8 and duplicate-JSON-member checks is prefixed

to every authoritative script, so the store rejects a malformed document rather than trusting the client that sent it.

Composition modes: A start-up gate distinguishes TEST, EVALUATION and PRODUCTION compositions and refuses to run a mismatched one. PRODUCTION and EVALUATION share one block of requirements and differ only in the connector endpoint, which makes “the measurements ran in a production-shaped composition” a property of the code rather than of this sentence. PRODUCTION is nevertheless unsatisfiable in this repository: there is no production vault and no production connector, and the gate fails closed for it. Every number in Section 6 was therefore collected in EVALUATION mode, with no test-only flags anywhere in the harness. The evaluation vault has two weaknesses relative to the production design, and both are declared rather than hidden: it shares the Redis trust domain, and its keys are operator-supplied rather than KMS-wrapped.

On test counts: The artifact carries a large automated suite and CI that fails if any test is skipped. We do not offer the test count as evidence of protocol assurance. A prior internal audit of this repository established that the suite had grown chiefly in rejection branches, gate scripts and artifact metadata, while the number of tests exercising the write-ahead path was close to unchanged; a reader who reads a doubled test count as a doubled protocol guarantee would be wrong. The evidence for the protocol’s behavior under faults is Section 6. The harness also found three defects the suite had missed, discussed in Section 9 because two of them would have biased results in AEP’s favor had they gone unnoticed.

Reproducibility: The environment is locked by a resolved dependency lockfile, the Redis image is pinned by digest, and CI provisions Redis from the same compose file the experiments use, so no job can pass against a differently configured store. Continuous integration runs four jobs: the full suite, the WAITAOF durability suite, a citation-range check over the formal model, and a gate that rebuilds the manuscript and re-derives every number in it from the frozen results. The build fails if any test is skipped.

6 EVALUATION

- RQ1** Under crashes, does AEP eliminate *undetected* duplicates, and what does its residual ambiguity depend on?
- RQ2** Which of the protocol’s two durability mechanisms produces which guarantee, and against which faults is each one observable?
- RQ3** What does the protocol cost, and how much of that cost is a choice?
- RQ4** How does recovery behave?

6.1 Setup

Systems: The seven of Table 1, implemented as thin variants over one connector, one workload driver and one oracle, so that only the differences differ. Each carries a machine-readable descriptor that tests check against the implementation, and a crash point that does not exist in a given system is marked inapplicable and never run rather than silently skipped.

What was delivered at after_barrier_before_dispatch: Table 3 names the position each crash point is meant to interrupt, and for two of the seven systems that is what was delivered. AEP-full and B3 resolve `after_barrier_before_dispatch` and `mid_dispatch` to different positions, and the kill at the first arrives before any byte is sent: in the crashed regime neither system records a dispatch attempt or an applied effect there. The five baselines resolve the two names to a single position, and the harness delivers that position by a watchdog that fires inside the socket wait. For B0, B1, B2, B4 and B4b the request was therefore sent, and the provider may already have applied the mutation, at both names. What those cells measure is a kill during transmission rather than a kill before it.

Those cells are therefore *excluded from every pooled baseline rate* in this section. The systems do not all pool the same number of crash points as a result: AEP-full and B3 pool six, B4 and B4b pool five, and B0 to B2 pool four, the fourth also being unavailable to them because `after_intent_before_barrier` cannot occur in a system that writes no intent. Table 7 carries the same statement in its caption. Nothing is excluded from AEP-full or B3, whose kill at that position was immediate.

The position itself was re-collected at the intended instant, and Section 6.6 reports it as a separate session.

Faults: Two fault sources run in every cell, and the first is not a regime. The provider times out on 15% of requests and returns a 5xx on 5%, answering the rest after a constant delay. It never returns a duplicate of its own (0%), so every duplicate the oracle records was applied by a caller that called twice. One code path in the runner applies this configuration to every run, which means it holds in every regime below, including the crash-free one.

The second source is the worker. Workers are separate OS processes killed with real `SIGKILL` at one of the six crash points of Table 3. A *regime* is a named crash or store fault, not a matrix dimension, and we report each separately because they are different experiments. Five are reported, and each is named here with the section that reports it:

- **crashed:** every execution is killed at the cell’s crash point. This is the regime RQ1 is about (Section 6.2), and it is also the regime of the detection ablation (Section 6.3.1).
- **crash-free** (p0): no worker is killed and the store is left alone. The provider faults above still apply. The only regime RQ3 may use (Section 6.4).
- **redis-kill-preack:** no worker crash; Redis is hard-killed with `docker kill -s KILL` at the instruction boundary between the intent CAS and the barrier acknowledgment, then restarted and verified. This is RQ2’s ablation (Section 6.3.2).
- **redis-kill-inflight:** the same fault armed at the last instruction before transmission and delivered afterwards, so that it lands once both arms have dispatched. It is the control for `redis-kill-preack`: where the fault falls on the far side of the branch point, the arms should tie, and they do (Section 6.3.2, under *Where the fault lands after the branch point*).
- **write-loss-preack:** no worker crash and no Re-

dis kill; the block device under Redis's append-only file stops accepting writes while reporting success, armed at the intent checkpoint. This is the fault the durability claim is defined against, and the regime that refuted our prediction (Section 6.3.4).

A sixth regime, a 30% crash probability, was collected and is in the artifact; no table here quotes its rates, so it is not one of the five.

The timeouts change how every rate below should be read. A timeout leaves the caller unable to say whether the effect was applied, which is the situation this protocol exists for, and a caller that retries can apply the effect twice with no crash involved. The four retrying baselines do: in the crash-free cells they duplicate at 0.1333–0.1800 over 150 executions each, where AEP-full, B3 and B4b record none. Every pooled baseline rate below sits on that background. Where the kill lands after the request is on the wire the rate approaches one; where it lands before transmission it falls back toward the background (Figure 2). A low per-cell rate is therefore evidence about the timeouts and not about the crash.

Oracle: A standalone mock provider records every *actually applied* mutation in a SQLite ledger (WAL, synchronous=FULL, one transaction per mutation). Duplicates are counted from that ledger, never from the system's own opinion of itself. The analysis is permitted to read exactly two files per run, the event log and the oracle ledger, and a source gate that parses the analysis modules fails the build if they import anything that could reach live protocol state.

Statistics: Rates are over executions. Per-cell rate intervals use a seeded run-cluster bootstrap (10 000 resamples); the B3–AEP ambiguity difference uses a stratified run-cluster bootstrap that preserves crash-point, endpoint-capability, and keying strata. Fisher's exact tests compare execution counts and therefore do not adjust for run clustering; we report them descriptively, not as evidence of equivalence. Seeds and repetition counts are recorded per run.

Two reporting rules we hold ourselves to: First, *no pooled table*. A rate computed across fault regimes is a property of how many runs of each kind were collected, not of any system; our own analysis tool prints a warning whenever more than one regime is present, and every number below names its regime. Second, *no absolute timing from an ungated host*. A run contributes a duration only if the operator declared before the run that the host cannot suspend *and* the run's wall-versus-monotonic divergence stays within tolerance. Both halves are necessary: a host that merely was not idle long enough to suspend is indistinguishable from one that cannot. Under this gate, zero runs collected before this rule existed contribute a timing number, and none does. Both rules were adopted after the first collection: the pooling rule when a mid-study review found the pooled summary table unusable as a source, the timing gate after a host suspension was discovered mid-run. Every rate and duration below is reported under them.

6.2 RQ1: undetected duplicates and the shape of the residual

Table 7 is the anchor result; the supplementary material shows the same executions pooled to one bar pair per system.

Three readings, in the order they matter.

AEP's two silent columns are empty, everywhere measured: Across 180, 180 and 180 crashed executions on the three capability classes, AEP-full recorded an undetected-duplicate rate of 0.0000, 0.0000 and 0.0000, and a lost-effect rate of 0.0000, 0.0000 and 0.0000. Every one of those executions was killed by SIGKILL at a named point around the external call. The matrix runner halted on this condition: a single undetected duplicate in AEP-full stops the matrix, a rule carried in the runner before any of these cells were collected. We do not call it a pre-registration. The governing document at the time described a contrary result as a defect to fix rather than a finding to report, and this project's pre-registration practice began later; what the runner gives is a stopping rule fixed in advance, not a commitment about how the outcome would have been written up. It did not occur.

The baselines without a pre-dispatch record duplicate in most crashed executions: B0, B1 and B2 sit between 0.74 and 0.78, and the weakest of the three comparisons against AEP-full is significant at $p = 9.4 \times 10^{-155}$ by Fisher's exact test. The gap between B1/B2 and B0 is small and that is the finding: a lease and a fenced CAS are doing exactly what they promise and neither promise is about the external call (Figure 2).

A durable log is not enough, and the two configurations of one engine land on two different corners: B4 has a write-ahead record, acknowledged durable by the same WAITAOF barrier AEP uses. It is not ablated on durability. It duplicates at 0.5067 on POS-ONLY and 0.4800 on NONE (150 and 150 executions), and it duplicates on AUTH as well, at 0.4467 over 150 executions. The endpoint's reconciliation capability does not rescue it. A read-back tells a recovering workflow what the provider currently holds, and B4's duplicate is a second application it chose to make, not a fact it failed to look up.

B4b is the same engine with Maximum Attempts set to 1, and it is the cleaner half of the demonstration: over 150 and 150 executions it records zero undetected duplicates on both classes and loses the effect in 0.4267 and 0.4467 of them instead, and 0.4600 over 150 on AUTH. One configuration line moves the engine from one silent corner of Section 1.1 to the other, and neither configuration reaches the third: across all 30 B4 and B4b cells the declared-ambiguity rate never exceeds 0.0000.

The engine itself was run, and it separates what B4 licenses from what it does not: We ran the production engine against the same non-cooperative endpoint, in the two configurations B4 and B4b instantiate, at the one crash point both the engine and our model can be cut at. With unlimited attempts it duplicated at 0.1576 over 590 executions and lost effects at 0.0000; with Maximum Attempts set to one it lost at 0.1424 over 590 executions and duplicated at 0.0000. Neither configuration declared ambiguity in any of 118 non-void runs (0 and 0 declarations), so the third corner of Section 1.1

TABLE 7

The trilemma, measured. Every execution in every cell was killed at one of the crash points of Table 3; rates are over executions, pooled across crash points within one endpoint capability. The systems do not all pool the same number of them, and Section 6.1 says why. AEP-full and B3 pool six. B4 and B4b pool five, their `after_barrier_before_dispatch` cells excluded because the harness delivered that kill inside the socket wait rather than before dispatch. B0 to B2 pool four, for that reason and because `after_intent_before_barrier` cannot occur in a system that writes no intent. The per-crash-point rates behind every cell here are in the artifacts per-cell metrics. AUTH/POS-ONLY/NONE are the reconciliation capabilities of Table 2. AEP-full and B3 — the same protocol with and without the durability barrier — are the only systems that record any declared ambiguity, and the only two whose undetected-duplicate and lost-effect columns are zero throughout. Their equality in every cell shown is this paper’s ablation result rather than an accident of this table, but the evidence for it is a bound obtained by pooling these columns, not by testing the cells individually: Section 6.3.1 puts both zero-event rates in $[0, 4.77\%]$ at joint coverage of at least 80%, four simultaneous bounds being two metrics on two systems. The per-class cells this table is organized by are not separately bounded — at that scope the width would be 20.1 percentage points — so what the table shows per class is no observed difference, and Section 6.3.1 is where the claim is quantified. Source: the artifacts per-cell metrics, crashed regime only.

system	undetected duplicate			lost effect			declared ambiguity		
	AUTH	POS-ONLY	NONE	AUTH	POS-ONLY	NONE	AUTH	POS-ONLY	NONE
B0 naive retry	0.7833	0.7667	0.7417	0.0167	0.0167	0.0083	0.0000	0.0000	0.0000
B1 lease-only	0.7750	0.7750	0.7833	0.0000	0.0083	0.0083	0.0000	0.0000	0.0000
B2 CAS-only	0.7833	0.7583	0.7417	0.0083	0.0083	0.0000	0.0000	0.0000	0.0000
B4 durable, ∞ attempts	0.4467	0.5067	0.4800	0.0133	0.0000	0.0133	0.0000	0.0000	0.0000
B4b durable, 1 attempt	0.0000	0.0000	0.0000	0.4600	0.4267	0.4467	0.0000	0.0000	0.0000
B3 intent, no barrier	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3667	0.7167
AEP-full	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0000	0.3500	0.7222

goes unreached by the engine itself and not merely by our model of it.

Each configuration lands in the corner the supplementary comparison assigns it, with exactly zero events of the other kind. The provider produced those events, not the crash: the engine’s cell kills one worker per ten-execution run, and its 0.1576 sits at the provider’s 15% timeout rate. The result is therefore about what a durable-execution engine does with an ambiguous call, which is the case this paper is about, and not about how it recovers from a crash. We do not set its rates against B4’s or B4b’s: ours kills every execution, so the two cells differ in exposure and in how many executions meet a timeout at all, and a cell matching both the position and the exposure has not been collected. **B4’s rates are our model’s, not the product’s**, and no rate in this paper should be read as an engine’s.

The collection itself took three attempts, and the record is in the artifact. The first was voided; the second was replaced because every run shared one fault seed; the replacement was registered before it was collected. Only the third is reported here.

Four things bound this and none is removed by having run the engine. The comparison is incomplete: the engine has no position answering to AFTER-INTENT-BEFORE-BARRIER, because its pre-dispatch record is written by the server inside one RPC, so there is no worker-side instant between the write and its acknowledgment. That is a hole exactly where B4 and B4b have data at all three capability classes. The rates are conditional on a 4000 ms Start-To-Close timeout that B4 has no analogue for; a different timeout is a different cell, and no timing number here is any engine’s recovery latency. The interval arithmetic is asymmetric: B4’s intervals rest on three runs against the engine’s thirty, and three clusters put resampled means on multiples of one thirtieth. And the crash window at the dispatch point is sized by the server and by loopback rather than by the protocol, so its width is not like-for-like with B4’s.

That is the sharpest form of the argument the paper

can make *against our model of a durable-execution engine*. That qualification carries the argument, and the paragraphs above are why: B4 is our implementation of the pattern, not a product, and we registered before collecting B5 that a large gap between the two would oblige us to re-scope every B4 claim. The argument is nonetheless made against a system that has everything the write-ahead pattern is supposed to provide. The record is necessary and it is not sufficient, because the record cannot contain the fact that matters, which is whether the provider applied the effect. What separates B3 and AEP-full from B4 is not durability but the absence of an edge back into the dispatching state: the transition table has no path that re-enters ABOUT-TO-FIRE, checked inside the atomic script (Figure 1). What separates them from B4b is that declining to re-send is not the same as knowing whether to.

6.2.1 The residual is set by the endpoint, not the protocol

AEP’s declared-ambiguity column is 0.0000 on AUTH, 0.3500 on POS-ONLY and 0.7222 on NONE. That ordering is not a defect that better engineering would remove; it is Table 2 showing through. Where the endpoint can prove absence, recovery resolves everything and nothing is left ambiguous. Where it can only confirm presence, absence is uninformative and the correct answer is “I do not know”. Where it cannot be queried at all, no amount of protocol recovers the fact.

The POS-ONLY rate depends on how the read-back is keyed, and we did not expect it to: The read-back in the cells above is keyed by the caller’s own reference. We pre-registered an alternative keying, an oracle fingerprint of the request, as a sensitivity check, and predicted it would leave every headline rate inside a ± 5 percentage-point margin. On POS-ONLY it does not: declared ambiguity is 35.00% under the caller’s reference and 44.33% under the fingerprint, a difference of 9.33 percentage points, and the margin was 5.

The two columns that carry the safety claim are unmoved: undetected duplicates and lost effects are zero un-

der both keyings. What moves is the residual, and the consequence is a limit on generality rather than on correctness. The POS-ONLY ambiguity rate is a property of the protocol *and* of how the recovering worker phrases its question, so the figure quoted above should be read as this keying's, not as the class's. AUTH is unaffected, because a read-back that can prove absence resolves the execution whichever key it is asked under.

The supplementary material plots the same executions as one bar pair per system, pooled over crash points and capabilities: the three systems on the right have an empty left bar, and only AEP-full and its ablation B3 have a right bar at all. The figure adds no number this section does not state; the bars are Wilson intervals on pooled counts, and the cluster-aware bootstrap intervals in the artifact's per-cell metrics are wider and are the ones to quote.

The rate decomposes further by crash point, and that is where the protocol's conservatism becomes visible; the supplementary material gives the full decomposition across all six crash points and three capability classes.

Two cells carry the argument. At *before_intent_write* the rate is zero in every capability class: the crash precedes the intent, nothing was promised, and there is nothing to be ambiguous about. At *after_barrier_before_dispatch* it is at its maximum, and it is equally at that maximum at *after_intent_before_barrier*: both read 30 of 30 under NONE and POS-ONLY, and zero under AUTH. At both, AEP-full's worker died before sending anything, so its own attempt left no effect behind. That is a property of the attempt and not of the crash point: a system that re-executes after the crash can still reach the provider from there, and Section 6.6 reports four baselines doing so. AEP declares ambiguity most often exactly where its own attempt did nothing. That is the false-positive ambiguity of Section 4.3, and its direction is the whole design: the protocol's errors are toward saying "I might have done something" when it did not, never toward silence when it did. The cells where an effect *does* typically exist, *mid_dispatch* and *after_response_before_resolution* under POS-ONLY, show low ambiguity, because a positive read-back confirms the effect and the intent resolves.

6.2.2 Why the provably-empty cell is not resolved

The *after_barrier_before_dispatch* row invites an obvious question: if the worker died before sending anything, *the system could know that*. It cannot, and the reason is the protocol's rather than the implementation's: nothing durable distinguishes that history from one where the worker died *during* transmission with the request already on the wire. Recovery sees an authorized, unresolved intent in both, so it fails closed. **The cell is empty in the world and unresolvable from the store.** The supplementary material, under *Why the provably-empty cell is not resolved*, gives the probe detail and prices the third barrier that would split the two cases.

6.2.3 The cost of the third corner

Declared ambiguity is not free, and a paper that presented it as a pure win would be selling something. Each declared ambiguity is an execution that has stopped, that will not

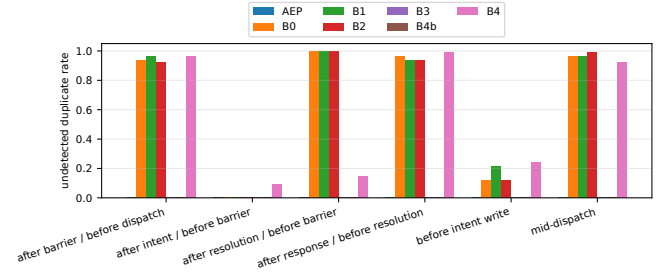


Fig. 2. Undetected duplicate rate by crash point, crashed regime, pooled over endpoint capabilities weighted by executions. The baselines' rate is near one wherever the kill lands once the request is on the wire, which is why a mechanism acting only *after* the call cannot help. At *before_intent_write*, where nothing has been sent, it falls back to the timeout background of Section 6.1.

proceed without a human, and whose resolution requires someone to establish out-of-band what the endpoint will not say. On a NO-READBACK endpoint under a crash-heavy regime that is most of the crashed executions. What we claim is not that this is cheap but that it is the price of the information the endpoint withholds, and that the alternative is not fewer incidents but the same incidents discovered later by a different department. We have not evaluated whether operators handle declared ambiguities well, or how much a real deployment would generate; both are stated as gaps in Section 9.

6.3 RQ2: which mechanism produces which guarantee

AEP contains two durability mechanisms and the literature usually treats them as one. The first is the *pre-dispatch record*: an intent, written and fenced, that exists before the external call does. The second is the *barrier*: the WAITAOE round trip that makes the record's durability a checked precondition of dispatch authority (Section 4.3). B3 is the ablation that separates them: AEP-full with the barrier removed and nothing else removed. Running it against three different fault classes assigns each guarantee to the mechanism that actually produces it.

The assignment is not the one we expected, and it is the paper's central structural result.

6.3.1 Detection is the record's, not the barrier's

Everything in Section 6.2 survives the barrier's removal intact: zero undetected duplicates, zero lost effects, and a residual of declared ambiguity shaped by endpoint capability.

Over 540 crashed executions per arm, B3 and AEP-full show no observed difference in either silent-failure metric. Both record an undetected-duplicate count of 0 and a lost-effect count of 0. Declared ambiguity is 195/540 for B3 and 193/540 for AEP-full; per capability class the two systems track each other: 0.0000 against 0.0000 on AUTH, 0.3667 against 0.3500 on POS-ONLY, 0.7167 against 0.7222 on NONE.

For ambiguity, B3 minus AEP-full is 0.37 percentage points. The stratified run-cluster 90% interval is [-1.11, 2.04] percentage points, which lies within a ± 5 percentage-point margin. We introduce that margin in this revision; it was not preregistered, and we state it as a stipulation rather than

TABLE 8

The barrier ablation on the detection metrics. B3 is AEP-full with the WAITAOF barrier removed and nothing else removed. Crashed regime; rates are over executions, pooled across crash points within one capability class. The p-values are execution-level Fisher tests over all capability classes; they do not account for run clustering and are not used as equivalence evidence.

metric	capability	AEP-full	B3	p
undetected duplicate	AUTH	0.0000	0.0000	1.00
	POS-ONLY	0.0000	0.0000	
	NONE	0.0000	0.0000	
lost effect	AUTH	0.0000	0.0000	1.00
	POS-ONLY	0.0000	0.0000	
	NONE	0.0000	0.0000	
declared ambiguity	AUTH	0.0000	0.0000	0.95
	POS-ONLY	0.3500	0.3667	
	NONE	0.7222	0.7167	

derive it. We do not claim it is immaterial to an operator: Section 9 records that we have not evaluated declared ambiguity as an operational outcome, and that the artifact has no escalation mechanism, so we have no basis on which to price one. The result supports equivalence under that stated, post hoc margin, but only for this fixed design. Each arm has 54 run clusters distributed over 18 matched strata, three runs per stratum, so the cluster-aware interval is a sensitivity analysis, not a strong general equivalence result. The execution-level Fisher value ($p = 0.95$) is not used as equivalence evidence.

A word on what the zeros can and cannot support. Fisher’s exact test on 0/540 against 0/540 returns $p = 1.00$ for duplicates and $p = 1.00$ for lost effects. Neither p -value carries information: a test between two zero counts has no power, and failing to distinguish them is not evidence that they are the same. What two zeros do support is a *bound*. The unit for that bound is the run, not the execution: each arm’s 540 executions are 54 runs of ten, and ten executions that share a run, a crash point and a system are not ten independent trials. The one-sided 95% Wilson upper confidence bound on a zero numerator over 54 run clusters is 4.77%. This is a one-sided bound using the 95th normal quantile; it is not the upper endpoint of a two-sided 95% Wilson interval, which uses the 97.5th quantile. Each system’s rate is individually bounded this way. When the two system bounds are stated simultaneously, Bonferroni gives joint coverage of at least 90%, not 95%. Under that simultaneous event, both rates lie in $[0, 4.77\%]$, so their absolute difference is at most 4.77 percentage points. This finite-sample bound is the support for “no observed difference” in the two zero-event metrics; the corresponding Fisher values are not evidence of equivalence.

Treating the executions as independent would give 0.50%. We use the run, for the same reason the Fisher values in Table 8 are not read as equivalence: with no events observed, nothing in these data estimates the within-run correlation that execution-level independence would require. The choice of unit decides nothing claimed below: the baseline duplicate rates this bound is contrasted against are 74–78%, more than an order of magnitude from either figure, so the conservative unit is taken because it is the

defensible one and not because the comparison needs it.

We report this as a finding rather than as a limitation, because it reassigns our own headline claim. The property that distinguishes AEP from B0, B1 and B2, and the property the paper is named for, is produced by the pre-dispatch record and by the transition table that refuses to re-enter ABOUT-TO-FIRE. It is not produced by waiting for `fsync`. A reader who wants the detection guarantee and not the barrier’s cost can have it under the faults measured here, and the supplementary material prices that configuration explicitly.

This also sharpens what B4 shows. B4 has a durable record, acknowledged by the same barrier, and duplicates at the highest rate in the study; B3 has a durable record, no barrier, and duplicates at zero. What separates them is not durability but that the record is written *before* the call and that no path re-enters the dispatching state. Both kept their record throughout, which is the condition the comparison assumes.

That condition is the scope of this result, and our own model says what lies outside it. Every detection cell we collected either left the record readable or never restarted the store. Table 6 carries the matched pair: with the barrier ablated and a restart that replays a shorter log, *no lost effect* fails; with the barrier enabled under the same restart, only version monotonicity breaks and *no lost effect* holds. So the model predicts that under a fault which destroys the record the barrier is what separates them, and that detection without it fails. We have not tested that prediction. It is falsifiable, the cell that would decide it is pre-registered, and it has not been collected.

6.3.2 Prevention is the barrier’s, and it needs the right fault

If detection does not need the barrier under these faults, what does the barrier do? It withholds the external call when the durability of the intent can no longer be confirmed. That is a different guarantee, it is worth stating in its own metric, and no worker-crash fault can exercise it, which is why Section 6.3.1 returns a null. The fault that exercises it is loss of Redis itself.

The metric is the *unwanted-applied-effect rate*: the fraction of executions in which a real, non-idempotent mutation reached the provider while the durability of its own intent record could no longer be established. It is not the same quantity as an undetected duplicate and it is not measured on the same regime; keeping the two apart is what this subsection is for.

We hard-kill Redis at `after_intent_before_barrier`: the instruction boundary between the intent CAS and the acknowledgment. AEP-full is inside WAITAOF at that moment; B3 is not, because B3’s ablation is exactly that round trip. One execution per run, no worker crash: the fault under study is Redis dying.

The controlled fault: The result below is collected under a *controlled* injector, and that is the change that makes it worth reporting. `docker kill` alone lands 368.4 ms after it is issued on this host; freezing the container first with `docker pause` and killing it afterwards lands in 58.3 ms, with a third of the spread, and leaves the container in the same end state. The fault class does not change, since it is the same F3 crash-stop at the same instruction boundary, so

the two are comparable, and the injector's own contribution to the race is narrowed by measurement rather than argued away.

Over 3 sessions and all 3 capability classes, AEP-full's unwanted-applied rate is 0.0112 (3 of 269 executions) and B3's is 0.9778 (264/270), on the same injected fault. The between-session spread in AEP-full's count, which a single collection cannot see and which the earlier version of this result was criticized for not reporting, collapses to 1 count per class across the 3 sessions. B3 stays at its ceiling throughout, never below 29 of 30 in any class in any session, which is what rules out the reading that the injector simply disabled both arms.

The result fell below its own pre-registered bound, and we report that as a surprise rather than a success: The prediction was registered before the data existed: 0.058, bounded [0.045, 0.081], derived from the measured landing distribution as the fraction of a 1 000 ms `WAITAOF` window the injector leaves open. The observed 0.0112 is below that bound. The pre-registration fixed in advance what a rate outside the bound would mean, namely the protocol behaving unexpectedly and not the mechanism succeeding, and we honor that here rather than presenting a lower-than-predicted number as a better result.

The reason the bound was wrong is measurable, and it is not calibration: the model behind it fails on two independent premises that push in opposite directions. First, it treats the exposure window as the injector's bench-measured landing latency. That does not survive contact with a collection: 106 of 240 uncontrolled AEP-full runs received a genuine `WAITAOF` acknowledgment *after* the bench landing had supposedly passed, and a `WAITAOF` acknowledgment requires a live server. Timed during the collections themselves, the same injector call takes 1 038.5 ms against the bench's 368.3 ms, a factor of 2.8 $\times$, with ranges that do not overlap. Second, it treats the `WAITAOF` wait as uniform on $U(0, 1\,000)$ ms. Measured over the runs the model was calibrated against, the fraction of waits under 58.3 ms is 0.0042, where uniformity assigns 0.0580, which is 13.9 $\times$ too generous, and worse the narrower the window. The two errors partly canceled at the uncontrolled window and could not at the controlled one.

The calibration that appeared to license the bound compared the wrong two quantities. The model predicts *dispatch*; the estimand is an *applied effect*, and dispatch is necessary for one without being sufficient. On the same runs the dispatch rate is 0.596 and the applied rate is 0.546. Against the quantity it actually predicts, the model was already wrong on its own calibration data. We record this because a pre-registered bound that agrees for the wrong reason is worse than one that disagrees: a disagreement is visible in the result, and an agreement reached for the wrong reason is visible only if someone checks.

What this does and does not license us to say: We report a rate and its mechanism, not a guarantee. **We do not claim that AEP-full prevents the unwanted effect.** The residual is non-zero and is irreducible under this fault class: freezing Redis synchronously at the checkpoint would make `WAITAOF` unanswerable, but B3 reaches the same checkpoint and still needs two further Redis calls, so the same freeze would stop B3 dispatching too and the contrast

would vanish because the injector had disabled both arms. What the controlled fault buys is that the residual is now bounded by something we measured rather than by this host's `docker` latency. As above, the bound we derived from that measurement was itself wrong in a way the data can show. The magnitude is not a constant of the protocol and this paper does not offer it as one.

The uncontrolled cell replicates the direction, and is not pooled with the controlled one: The original collection used `docker kill` alone, and is kept as the uncontrolled replication. It is reported separately rather than pooled: the two injectors have different landing distributions, and combining them would trade a stated limitation for an unstated confound. There the rate is 0.3333 for AEP-full against 0.9333 for B3 (Fisher's exact test, two-tailed, $p = 1.9 \times 10^{-6}$), a difference of 18 executions on the same injected fault, the same direction as the controlled cell, at a magnitude the uncontrolled injector sets. This is the only fault in the study under which the two systems diverge; everywhere else they differ by at most 2 executions in 540 (Table 8). Both systems declared ambiguity on all 30 runs: the endpoint is `NO-READBACK`, so neither can resolve what it does not know, and both fail closed. The difference between them is not in what they *say*, which is the detection guarantee that Section 6.3.1 showed does not depend on the barrier under these faults. It is in what they did to the outside world before saying it.

The direction replicates; the magnitude moves, which is why the controlled cell exists. The cell was re-collected 4 further times under an identical configuration, and AEP-full's unwanted-applied count ranged 4–20 out of 30 while B3 stayed at 28/30 in every one. Taking the session as the unit, the difference averaged 17.2 effects, 95% interval [6.1, 28.4]; against that spread the controlled cell's 1-count per-class spread over 3 sessions is the change worth reporting. Two negative findings travel with the cell and are not softened by being moved: attributing the movement to kill latency did not succeed at this design's precision, and B3 is not the negative control an earlier version of this paper reported it to be, because its own session-level interval is far too wide to have contradicted the mechanism whatever it had done. The supplementary material carries the per-session tables for both: the uncontrolled counts, the four session-level kill-latency differences with their intervals, and the same analysis for B3. The scope of the cell is one pre-acknowledgment crash point and one host, and the `AUTHORITATIVE-READBACK` class was subsequently collected over 4 sessions at a precision inadequate to the question.

Where the fault lands after the branch point, the arms tie: The same fault delivered *after* dispatch separates nothing, and we collected that rather than inferring it. Arming the kill at the last instruction before transmission and delivering it 200 ms later puts it in flight, after both arms have already dispatched. Over 2 sessions the two arms differ by at most 1 count in any capability class, and both sit at their ceiling. Two qualifications belong with that number. The second session is a deterministic replay of the first rather than an independent draw. The harness assigns seeds per cell and repetition, so where no race remains the second collection reproduces the first exactly, which is itself

the clearest evidence that no race remains. And B3's lowest cell is 27/30, exactly the floor below which the result would have been read as a mis-timed injector rather than a tie. The result sits on that boundary, with no margin above it.

What is structural, across all three faults, is that the race exists for AEP-full at all and cannot exist for B3.

6.3.3 The barrier's durability claim, under the fault that can actually test it

There remains a third thing the barrier is supposed to do, and it is the one most readers assume it does first: protect the write-ahead record from being lost when Redis dies. We tested that premise before building on it, and for the fault everyone reaches for first, it is false.

10 trials, each phase-aligning the `everysec` timer with a `WAITAOF` so the window under test is as wide as it can be, writing a key *without* waiting for it, hard-killing the container with `docker kill -s KILL`, restarting, and reading back. Every kill landed (every container came back with its `uptime` at zero) and every kill landed inside the window (write-to-death 419–992 ms against a 1 000 ms `fsync` period). **Not one unfsynced write was lost: 0/10.**

The mechanism is not subtle once seen. `appendfsync everysec` defers the `fsync(2)`, **not** the `write(2)`: Redis writes the AOF buffer to the operating system on every event-loop iteration. A `SIGKILL` destroys the *process*; the kernel and its page cache are untouched, and a kernel that is still running flushes those bytes to disk. The same result reappears at $n=60$ inside the ablation cells above, where a canary write made immediately before each kill survived in 30/30 and 30/30 runs.

That narrows the claim to a named fault class: loss of the page cache, by host power failure, kernel panic or VM destruction. An earlier draft of this paper stopped there, naming a fault it had not injected. We now inject it.

Making the storage lose the writes: The probe is the one above with the fault swapped. A loop-backed block device carries a `dm-flakey` target whose `drop_writes` feature silently discards every write bio while serving reads correctly, which is what a storage stack looks like to a kernel whose writes will never reach the platter. Redis 7.2.5, the same build as the rest of the evaluation, runs with its append-only file on an `ext4` filesystem over that device. Each trial writes one key through `WAITAOF` and one without, switches the device into `drop_writes`, kills the server, unmounts to discard the page cache, restores the device and reads both keys back. The switch is the instant of power loss: what was `fsync`-ed before it is on the device, what was still dirty is not.

Two details decide whether this measures anything. `dmsetup suspend` freezes the filesystem and flushes outstanding I/O by default, either of which would push the unacknowledged write out before the fault took effect and produce a confident null; the reload runs with `-nolockfs -noflush`. And a self-test runs first on the device alone, with no Redis in the picture: a file `fsync`-ed before the switch must survive and one `fsync`-ed after it must not. The probe refuses to report Redis trials unless that holds.

The result: The probe ran 90 trials in 3 independent replications. Each replication builds the device stack from scratch, and each *trial* within it rebuilds the filesystem with

`mkfs.ext4` and starts a fresh server, so no trial can read a key a previous trial's append-only file still holds. **Separation was perfect in every replication:** the acknowledged record survived 30/30 and the unacknowledged one was destroyed 30/30 in each, 90/90 and 90/90 overall. Against the same two keys under a process kill, 0/10 were lost; the fault classes separate completely.

We give no p for either comparison, and the reason is that no unit supports one. At the trial level the test would assume an independence the shared device stack violates, and it would ignore the pairing, since both records are written in the same trial and meet the same write-loss event. At the replication level, three against three, the two-sided minimum a perfectly separated result can reach is 0.1, which is the design's floor rather than a statement about these data. The complete separation in every replication is the finding, and it does not need one. The filesystem was mounted `ext4 rw,relatime` with its journal enabled and no `nobarrier`, and the server's `appendfsync` was read back as `everysec` from the running instance; the probe refuses to measure if either is otherwise, because under `always` there is no unacknowledged write to lose and under `nobarrier` a surviving acknowledged one would prove nothing.

The barrier's durability benefit to the record is therefore real and now measured, and the fault class matters more than the mechanism. What that benefit is worth to a *dispatch* is the next subsection's question, and the answer there is not the one we expected. The Redis documentation's "you may lose 1 second of data if there is a disaster" [8] is correct; what our two probes add is that "disaster" must mean the host, that the widely repeated reading of the sentence as covering a crashed Redis *process* is wrong, and that once the host is the fault the sentence is exactly right.

One qualification, and it is the same one the process-kill probe carries: the exposure window is maximized by design. The unacknowledged write is issued 12.4–61.8 ms before the device stops accepting writes, inside a 1 000 ms `fsync` period that a preceding `WAITAOF` has just reset. 90/90 is therefore the loss probability for a power cut placed where it does the most damage, not the rate a uniformly-timed one would produce; that rate is bounded above by it and approaches it as the write approaches the start of the period. The design is deliberate and it is the same design that made the process-kill probe's 0/10 strong: both probes give the loss hypothesis its best case, and only one of them finds loss.

6.3.4 Running the protocol under that fault, and what it refuted

The probe above tests `WAITAOF`, not AEP. It runs two Redis keys against a device that stops accepting writes; it does not run AEP-full and B3 through the harness under that fault, and so it measures no difference in *protocol* outcomes. What it establishes is the premise the durability argument rests on. We then ran the extension, and it is reported here rather than as a caveat because it refuted what we predicted.

What was registered, and what happened: We pre-registered that AEP-full would withhold *dispatch* when its write-ahead record was destroyed and that B3 would proceed. A full session was collected with the harness's Redis running its append-only file on the flakey device: both arms, 30 runs each, 10 executions per run, `drop_writes`

TABLE 9

Median step latency, crash-free runs only, E5-gated, from the three-run cells the runs column reports. Increments are over B0, the no-protocol system on the same host and provider. The whole write-ahead protocol minus the barrier is the B3 row; the barrier is the difference between B3 and AEP-full. The intervals in Section 6.4 come from a later fifteen-run collection, so they are not this table's arithmetic.

system	runs	median step (ms)	over B0 (ms)
B0 no protocol	3	2010.2	—
B1 + lease	3	2014.0	3.9
B2 + fenced CAS	3	2017.4	7.3
B3 full protocol, no barrier	3	2038.2	28.0
AEP-full	3	4004.9	1994.7
B4b durable, 1 attempt	3	4013.4	2003.3
B4 durable, ∞ attempts	3	4015.7	2005.5

armed at the intent checkpoint, without a delivery failure. **The prediction is refuted.** AEP-full applied an effect in 285 of 300 executions and B3 in 287; both arms finished at ceiling, and neither withheld.

Why, and it is the more useful finding: **WAITAOF returned success for every durability acknowledgment requested after the device had stopped accepting writes**, 300 of them, with 0 failures. `dm-flakey's drop_writes` discards each write bio and reports success, so the kernel sees no error, `fsync(2)` succeeds, and Redis has no way to learn otherwise. The barrier withholds dispatch on a *failed or absent* acknowledgment; it was handed a successful one and dispatched, behaving exactly as specified on an input that was false.

The bounds, stated narrowly: **This is not evidence that the barrier fails to withhold under write loss**: the condition the experiment exists to create, the barrier meeting an acknowledgment that fails or never arrives, was never reached, so no run exercised withholding. **Neither is it evidence that the barrier works.** What it establishes is that the durability guarantee rests on an unstated premise, that a WAITAOF success means the record reached the device, and that under storage which discards writes *silently* the premise is false and the protocol cannot detect it. Storage returning EIO would let the acknowledgment fail and put the barrier back on the input its guarantee is defined over, so this result bounds the guarantee to storage that reports its own failures and says nothing about storage that does not. It is evidence about storage, not about the protocol. One host, one Redis version (7.2.5, `appendfsync everysec`), and we claim it for no other.

6.4 RQ3: cost, and how much of it is optional

Only crash-free runs on a suspend-disabled host contribute. The provider is configured with a constant 2000 ms delay, so one round trip is the floor and the interesting quantity is the increment over it.

The overhead is the barrier, and the rest is not separable from zero. A median step costs 4004.9 ms under AEP-full against 2010.2 ms under B0, which does nothing but call the provider. Almost all of that gap is one mechanism. At fifteen crash-free runs per arm the two WAITAOF round

trips cost 1939.7 ms between them, with a 95% interval of [1855.8, 1962.4] ms.

Everything the write-ahead protocol does apart from waiting for `fsync`, meaning the intent ledger, the vault, the request binding, the preflight and the recovery service, costs 33.9 ms [-122.6, 120.1] ms on a two-second call. That interval spans zero. B3's arm is a mixture of two modes, and restricting both arms to their lower mode gives 16.3 ms [-54.1, 47.5] ms, which spans zero as well. We report both because they were both pre-registered and they disagree, and choosing between them after seeing them is the move this paper is arguing against.

So the decomposition supports an ordering and not a partition. The barrier dominates: its interval is nowhere near zero and the residual's contains it under both readings. What cannot be said is what fraction of the protocol's cost is optional, because at this precision the non-barrier remainder is not distinguishable from nothing at all. An earlier draft of this paper quoted that remainder as a point figure of about thirty milliseconds; it rested on three runs per arm, and twelve more did not confirm it.

Read against Section 6.3.1 the ordering still says something stronger than "AEP costs two seconds". Whatever the detection guarantee costs, it is the difference between B0 and B3, since B3 is the system that already has that guarantee in full, and it is small enough that fifteen runs cannot separate it from zero. The 1939.7 ms buys the prevention guarantee of Section 6.3.2 and nothing else.

That figure is also a property of a *configuration*, not of AEP. Under `appendfsync everysec` an acknowledgment cannot return until the next scheduled `fsync`, and those are one second apart, so a barrier costs up to a second and the protocol pays that twice. The measured total sits near the top of that range rather than in the middle of it; the reason is unmeasured.

What the barrier costs, and the supplementary prices it: The $\approx$ 2-second figure above is the one a reader would carry away as "AEP's overhead". It is not: it is the price of one of the protocol's two guarantees, under one of the durability settings that guarantee can be bought at. The supplementary material, under *What the barrier costs*, gives the three measured points as a deployment faces them: `everysec` with the barrier, `always` with the barrier, and the barrier removed. Its table is built from the three-run cells, so its `always` column and the -9.2 ms quoted here differ in sign; they are two cells, not two readings of one. The claim this section rests on takes the later collections: the barrier costs 1939.7 ms under `everysec` over fifteen runs and -9.2 ms under `always` over fifteen, and both are stated above with their intervals.

6.5 RQ4: recovery

Recovery is reported as counts. In the crashed regime, every interrupted execution that had written an intent was resolved to a terminal classification: 450 of 450 for AEP-full and 450 of 450 for B3, over 5 crash points and 3 endpoint capabilities. The classes those executions resolved to are the ones Table 7 already reports.

The sixth crash point is counted separately, and it is not a recovery failure. At `before_intent_write` the worker

dies before any intent exists, so there is no record for recovery to act on (Table 3). The 90 executions there for AEP-full and the 90 for B3 resolve to no terminal classification at all. Pooling them with the rest would produce a figure that reads as a failure rate for a component that was never given anything to do.

We report no rate and no interval for this. The runs behind each cell resolved identically, so a bootstrap interval would be degenerate and would claim a precision they cannot support; the counts are the stronger statement. Absolute recovery latency is not reported either. Under the timing gate of Section 6.1, 176 of 432 runs contribute a duration, which leaves too few gated crashed runs for a distribution, and Section 9 records that as a cost of the gate. The baselines declare no recovery service and are excluded from these counts rather than scored; the supplementary material, under *RQ4: recovery*, gives that argument in full and carries the one reconciliation disagreement in 432 runs and what became of it.

6.6 A re-collected crash point, reported separately

Section 6.1 records that the five baselines' *after_barrier_before_dispatch* cells were delivered by a watchdog firing inside the socket wait, so what they measure is a kill during transmission. On 2026-09-24 we collected that position again with the kill delivered at the instant the name denotes: 15 cells, 45 runs, 450 executions, the five baselines across all three capability classes.

This is a separate session, and none of it is pooled with the rest of this section. The matrix was collected in early August. A rate computed across two sessions is in part a property of how many runs of each it contains, and this paper reports sessions separately for that reason (Section 9).

We registered a prediction and it was refuted. We predicted that an immediate kill at this position would leave applied effects, undetected duplicates, lost effects and dispatch attempts all at zero across every one of the 450 executions, because a worker killed before transmission sends nothing. Four of the five systems recorded all four quantities above zero.

The prediction was wrong about the execution rather than about the kill. It reasoned from the killed attempt, which does send nothing, to the execution, which in this regime outlives it: a supervisor re-executes, and the four systems configured to retry call again. Every applied effect in the collection has a recorded dispatch attempt behind it.

What the collection does establish is the position, and it establishes it three ways. The number of executions recording an applied effect with no dispatch attempt is 0 of 450; in the matrix's deferred cell that pattern appears in 82 of B4b's 90 executions, and it is the signature of a worker dying after its bytes have left. B4b, which is configured never to re-execute, applied 0 effects over 0 dispatch attempts in all three of its cells. AEP-full and B3, whose kill at this position was immediate in the matrix as well, record the same zeros there.

The duplicates in this cell are the provider's timeouts and not the re-execution, and we draw no duplicate-rate conclusion from it. A kill before transmission leaves one re-execution, and one re-execution applies one effect; every

duplicate here required a second dispatch inside the same execution, which is what a timeout produces. The rates are 0.0667–0.2667 per cell, against 0.1333–0.1800 for the same systems in the crash-free cells, where no worker is killed at all.

7 RELATED WORK

AEP composes established mechanisms: leases, fenced compare-and-swap, write-ahead logging, and an fsync barrier. We do not claim that durable intent plus locking is new. The contribution is a capability-specific recovery semantics for effects outside the coordinator's atomicity domain, and an evaluation that separates detection from prevention.

Prior work is therefore organized here by the three questions that decide whether it applies to a non-cooperative endpoint. **First, what does it require the endpoint to supply?** A mechanism may need idempotence, an idempotency key, enrollment in a transaction, or a compensating action, and one that needs any of these does not apply where none can be obtained. **Second, can it report *unknown*?** That is, does it have a terminal outcome meaning "an effect may exist and I cannot establish whether it does", as distinct from success or failure. **Third, what residual does it leave when its stated assumption fails?** The candidates are a duplicate effect, a lost effect, a blocked participant, a compensation gap, or a declared ambiguity. The three subsections below are the three questions, and every system discussed is placed against all three. The same placement is tabulated in the supplementary material, which lists fourteen mechanisms and AEP against these columns; of the fourteen, only AEP answers *no* to the first question and *yes* to the second.

7.1 What the endpoint must supply

Leases and fencing: Leases provide time-bounded mutual exclusion under crash faults [9], as an infrastructural service [12], as a coordination kernel for client-side locking [13], and in commodity form [14]. A lease holder may stall past expiry and write afterwards, so a monotonic fencing token is also required.

Chubby states both the hazard and the remedy in primary form, which matters because AEP's guard depends on the remedy and because the requirement it imposes is the one this paper cannot satisfy. A client holding a lock may issue a request and then fail, a second client may acquire the lock and act, and the first request may arrive afterwards. The remedy is a *sequencer*: the lock name, a generation number that changes on every transition from free to held, and the acquisition mode. The client passes it to the file server, *and the file server is expected to validate it* [12]. That validation requirement is the crux: a fencing token protects a resource only if the resource checks it, which makes fencing a property of the *resource* rather than of the lock service. Chubby concedes the point by shipping a fallback for servers that cannot be modified to validate sequencers: *lock-delay*, which merely withholds a lock for a bounded period after an abnormal release and which that paper calls imperfect. It is the closest prior art to AEP's setting and it is a timing heuristic, not a guarantee. Kleppmann's analysis remains the clearest statement of the failure mode in the commodity

Redis setting and is cited as the practitioner account it is [11]. B1 and B2 measure exactly this boundary: fencing protects the store, not an endpoint that never interprets the token (Table 7).

Logging, outboxes, and what compensation assumes: The expected-version CAS is optimistic concurrency control [15]. ARIES writes intent before an action that the same recovery domain can undo or redo [16]; AEP instead records an action performed by a foreign endpoint that may support neither operation. The transactional-outbox pattern likewise stores a message before a relay publishes it, but explicitly permits the relay to publish twice after a crash and therefore assumes an idempotent consumer [17]. B3 is structurally outbox-like, with a no-re-entry rule in the measured runner; it does not make the remote mutation transactional.

Compensation is the standard answer once atomicity is unavailable: a committed effect cannot be abrogated, only altered by a further transaction [18], formalized as compensating transactions that undo committed work without cascading aborts [19] and applied to long-running activities by pairing each step with a compensating one [20]. All three presuppose two facts about the effect: that it *happened*, and that a semantically inverse operation exists and may be applied. AEP's endpoint reliably supplies neither. In the NO-READBACK class the first is unobtainable by construction, and compensating for an effect that may not have occurred is itself a mutation with the same problem, which is why declared ambiguity is terminal here rather than a prelude to compensation. Helland describes the broader consequences of operating without distributed transactions and the centrality of idempotence [1], [21].

What an idempotency-key contract requires of the server: The expired IETF HTTPAPI working-group draft for an Idempotency-Key header documents the cooperative alternative, and because AEP's premise is an endpoint offering no such contract it is worth being precise about what one costs the server rather than treating it as a header. The draft places the burden squarely on the resource: it must manage the key's lifecycle, it *MUST* publish an idempotency specification including any expiration policy, and on a repeat it *SHOULD* respond with the result of the previously completed operation, success or error, rather than executing again [22]. Storing the key is the least of it; the server must also retain the *outcome*, scope the key, and decide how long the guarantee lasts.

The draft mandates little beyond publication, asking servers to declare a policy rather than prescribing retention or scope, and the deployed contracts show why that matters. Stripe saves the status code and body of the first request for a key and returns that same result on a repeat, including 500s, prunes keys after at least 24 hours, and rejects a reused key whose parameters differ [23]. Adyen returns the original response without reprocessing, scopes keys to the company account, does not cross-check duplicates across regional endpoints, and holds keys for seven to fourteen days [24]. The same header name therefore carries materially different guarantees: a retry protected at one provider is outside the window at another, and the caller cannot tell from the protocol. These are specifications and vendor documentation rather than research, and are cited as such. The claim is about what the contract requires, for which the

normative text is the primary source. AEP assumes the case beneath all of them: no key is interpreted at all. What the deployed contracts add is that even cooperation is bounded, so a caller's obligation to reason about an unsettled effect does not disappear where a key exists; it shrinks to the retention window.

Why not two-phase commit: The obvious objection is that atomic commit already solves this, so it is worth saying exactly what that would require of the endpoint. Under 2PC each participant durably logs its own state transitions to stable storage and answers the coordinator from that log during recovery [25]. A participant that has voted to commit then enters an uncertainty window in which it can neither commit nor abort on its own authority, and must block, holding the prepared state, until the coordinator is heard from again [26]. Both are obligations on the *participant*, and a legacy endpoint that accepts a request and returns a response offers none of them: no prepare, no vote, and no way to ask it to hold a decision open. The barrier is not that 2PC is expensive here; it is that the protocol has no participant to run.

Nor is this a counsel of despair about distributed transactions. Spanner shows the obligations can be met at global scale, with a 2PC implementation integrated with replication and concurrency control and a time API that bounds clock uncertainty [27]. It is, however, built by an operator who controls every participant and can equip each one. That is the condition AEP assumes away, and when enrollment is unavailable the burden moves to the caller and idempotence becomes the pivot on which correctness turns. AEP is what is left when the endpoint does not supply that pivot either.

7.2 Whether it can report *unknown*

Locks with intent, and the durable-orchestration line: Olive is the closest mechanistic antecedent. Its locks with intent durably associate code with a lock and provide exactly-once execution despite client failures and concurrent duplicate executions [28]. A write-ahead intent as a checked precondition of execution is therefore prior work, not AEP's novelty. Olive's guarantee is realized through operations made recoverable within its cloud storage abstraction; AEP starts where that atomicity scope ends, and consequently offers no exactly-once claim: it terminates in declared ambiguity and prohibits automatic re-dispatch.

The idea is older than the current generation of products. Workflow management systems already specified business processes and generated implementations that recorded progress durably across long-running activities [29]; the recovery semantics they relied on were the compensation models above, which is why sagas sit underneath that literature rather than beside it. What the current generation added is a sharper boundary, and each system states the same assumption in its own documentation. Cadence introduced the framing: workflow code is fault-oblivious and may not call an external API directly, only orchestrate *activities*, and because the service does not recover an activity's state, activity failure is expected and retried [30]. Restate journals each step and replays after a crash, skipping completed steps, and it names the residual exactly: if a side effect completes but the failure prevents its result being

recorded, replay runs it again, so side effects are required to be idempotent [31]. AWS states it normatively: a Step Functions workflow conforms to an at-least-once execution model in which each step may run more than once, and step-level idempotency remains the caller’s responsibility [32]; its durable execution SDK turns the trade into an explicit per-step knob, recommending at-least-once for idempotent operations and at-most-once for externally side-effecting ones such as charging a card or posting to a non-idempotent API [33]. That last sentence is this paper’s problem statement, written by a vendor. DBOS takes the other route, placing system state in a distributed transactional DBMS so that services are ordinary database operations, recoverable as such [34], with recoverability again purchased by keeping the effect inside the transactional domain, the same boundary Olive draws.

Four systems in this line draw that boundary in different places. Beldi extends Olive’s atomic logging to stateful serverless functions and adds a two-phase commit variant across them [35]; the atomicity covers Beldi’s own data structures and functions that speak its commit protocol, so an endpoint implementing neither is outside it. Boki gives functions a shared log and builds durability, consistency and fault tolerance on that primitive [36]; state living in the log is recoverable, and an effect performed elsewhere is not in the log to recover. Durable Functions replays the orchestration from its history and re-invokes activities, safe when the activity can be re-run [2]. ExoFlow is closest in spirit to this paper: it decouples recovery from execution and asks the author to annotate each task’s semantics, including non-determinism, so the system can choose a recovery strategy per task [3]. That is a declaration about the task by someone who knows it, whereas AEP’s declarations are about the *endpoint*’s capacity to answer after the fact, which a caller cannot annotate its way out of.

The axis is not feature coverage but what each assumes of the invoked step: that it is idempotent, compensable, or inside the same transactional domain. Where that holds these systems are correct and AEP is unnecessary; AEP’s endpoint satisfies none of the three. Temporal states the operational premise directly: activities should be idempotent because they may execute more than once [4]; setting Maximum Attempts to one selects the opposite retry policy [7], and the engine’s own account of when an activity is considered failed rests on timeouts and heartbeats rather than on any fact about the endpoint [37]. Our B4 and B4b controls instantiate these two policies against the same non-cooperative endpoint. They are semantic controls, not claims about those products’ complete implementations.

That distinction is no longer only a caveat. We subsequently ran the production engine itself against the same endpoint, in both configurations, at the one crash point the engine and our controls can both be cut at. Unlimited attempts duplicated and lost nothing, one attempt lost and duplicated nothing, and neither configuration declared ambiguity in any run, so each landed in the corner the supplementary comparison assigns it. Section 6.2 reports the rates, what produced them and the four things that bound them; we do not restate them here. Our measured comparison is with one engine and does not transfer to the others, and no rate in this paper should be read as a

product’s.

The ablation supplies AEP’s narrower empirical delta. B3 and AEP-full share the pre-dispatch record and no-re-entry transition but differ on the durability barrier. We observe no difference in their crashed-regime detection outcomes, bounded by pooling the capability classes: both zero-event rates fall within $[0, 4.77\%]$ at joint coverage of at least 80%, four bounds rather than two (Section 6.3.1). The Redis-kill experiment isolates a prevention effect. We found no prior evaluation that measures this detection/prevention decomposition with an independent external-effect ledger.

Agent execution reliability: Three contemporaneous systems overlap substantially and delimit the claim. Verified Tool Calls formalizes non-atomic agent tool failures and uses postcondition verification, verify-before-retry, and idempotency keys in a simulated fault environment [38]. Its verifier requires a queryable postcondition; without server-side idempotency, it characterizes the remaining race as best-effort. AEP evaluates the authoritative, positive-only, and absent-readback cases separately and uses terminal ambiguity for the last case.

Three contemporaneous systems bound the comparison. LogAct durably exposes intentions in a shared log before execution [39], broader in agent-state-machine structure where AEP is narrower: what can be concluded about one non-idempotent external mutation after an interruption. The Sovereign Execution Broker reserves certificate nonces before mutation and refuses retry after possible invocation absent durable or target-side evidence [40]; its mandatory broker boundary with cryptographic certificates is stronger than AEP’s trusted-process guard. ACRFence addresses semantic rollback during agent checkpoint-restore [41], adjacent rather than a basis for ranking which work is “nearest.” AEP’s distinct evidence is the three endpoint-capability classes, terminal declared ambiguity, real SIGKILL and Redis durability faults, and the B3/AEP ablation.

7.3 The residual when the assumption fails

Durability acknowledgments and what they are worth: AEP’s barrier withholds dispatch until the store acknowledges the intent durable, so the protocol is only as strong as that acknowledgment, and a body of storage work establishes that the acknowledgment can be false. Rebello et al. show that none of five widely deployed applications (PostgreSQL, LMDB, LevelDB, SQLite and Redis) handles *fsync* failure sufficiently, and that on Linux the affected pages are marked clean regardless of whether writeback succeeded [42]; the operational discovery that a successful *fsync* does not mean the data reached persistent storage was made in PostgreSQL and reported at the time [43]. Ganesan et al. find that eight distributed storage systems, Redis among them, do not consistently use redundancy to recover from file-system faults, so a single fault can produce data loss or unavailability [44]. Alagappan et al. show that recovery ignoring protocol-level knowledge loses data in replicated logs, and repair it for Raft-style [45] systems [46].

We place our own durability result in that line rather than beside it: Section 6.3.4 reports a barrier that dispatched on an acknowledgment which was already impossible to honor, because the underlying device discarded the write

and reported success. The protocol behaved exactly as specified on an input that was false. What this literature supplies is the reason that is a property of the storage stack rather than a defect peculiar to our implementation, and the reason a barrier's guarantee must be stated as conditional on the acknowledgment it rests on.

Exactly-once: state, delivery, and effect: The paper declines the term "exactly-once," and should say what it is declining, because the term covers three different things. Consistent recovery for distributed computation is classically obtained by snapshotting global state [47], and modern dataflow engines make that practical by injecting barriers into the stream and persisting operator state asynchronously [48]. What those mechanisms guarantee is that *internal state* is restored to a consistent cut; an external mutation already performed is not inside that cut and is not undone by restoring it.

Exactly-once *delivery* is not on offer from any system, and the obstacle is epistemic rather than engineering. For a sender and a receiver to agree that a message was delivered, each acknowledgment requires its own acknowledgment; Halpern and Moses formalize this as the unattainability of common knowledge when communication is not guaranteed, and connect it to the coordinated-attack problem [49]. No protocol removes that regress. What systems provide instead is exactly-once *processing*, obtained by enrolling the destination: Kafka couples an idempotent producer protocol with transactions so that output records, offset commits and state updates commit atomically, and the read-process-write cycle either takes effect once or not at all [50]. That guarantee is real, and it is real *because* the sink is inside the transaction, which is also why the snapshot mechanisms above are paired with a transactional sink.

This is the distinction the paper needs and the one the term usually obscures. A message delivered once is not an external mutation performed once. Where the destination can be enrolled (a Kafka topic, a database in the same commit scope), exactly-once processing is available and AEP is unnecessary. Where it cannot, no retry discipline on the caller's side reconstructs it, and the residual is not a weaker delivery guarantee but an unanswered question about an effect. This is also the end-to-end argument [51] in an uncomfortable form: the check that would settle whether the effect happened belongs at the endpoints, and the endpoint that would have to perform it is exactly the one that cannot.

Fault-injection lineage: AEP's named crash boundaries and independent oracle follow a mature crash-consistency methodology [52], [53], [54], [55], which constructs post-crash states, injects power faults, or specifies a system's crash model as litmus tests. The last is unavailable here in the direction that matters: a legacy endpoint publishes no crash-consistency model to write litmus tests against, which is the problem rather than an oversight.

Three distributed-systems methods bound the comparison. FATE explores *sequences* of failures against declarative recovery specifications [56]; our cells inject one fault at one named boundary, trading coverage of combinations for an exact statement of where the single fault landed. Lineage-driven injection reasons backwards from a correct outcome to the faults that could have prevented it, over a model of dropped messages, failed nodes and partitions [57], a

model that excludes crash recovery, the class this paper is about. Elle infers a dependency graph from what clients saw [58]; the contrast is the oracle, since our provider ledger is written independently of the system under test and is never consulted by it. AEP applies this lineage to a caller-to-endpoint protocol, and the harness is an evaluation instrument rather than a claimed contribution.

Read down the three columns, the position is this. AEP's claim is not the invention of write-ahead intent, locking, or fail-closed interposition. It is the measured composition for endpoints with authoritative, positive-only, or no readback: capability-specific recovery; a terminal declared-ambiguity outcome when no query can settle the effect; real process and durability-fault evaluation with an independent ground-truth ledger; and an ablation that assigns detection to the pre-dispatch record plus no re-entry, and prevention to the durability barrier.

8 ARTIFACT AVAILABILITY

The public repository available via the submission system contains the implementation, evaluation harness, mock provider, baseline systems, analysis pipeline, manuscript source, and the tracked derived analysis products used by the number generator. It does *not* contain the raw run directories, which are bulk data: those are carried by a separate archive, available via the submission system.

The raw evidence archive: Two archives under one DOI: the 2026-09-03 build (20 collection roots, 1458 run directories, 26300 files) and a 2026-09-15 extension covering the trees it did not reach (11 roots, 1332 run directories, 18494 files). That is 2790 run directories and 44794 files in all, 848MB uncompressed and 42MB compressed. Between them they carry the 432-run evaluation, the `appendfsync=always` arm, the deployment sweep of Section 6.4, the real-Temporal baseline, the write-loss cell, the voided oracle-disagreement run, every prevention replication, and the arms voided for having run against the wrong container runtime. Every file carries a SHA-256 in its own archive's checksum manifest, and that manifest's own digest (87fa2d53... and 54d1ab0f...) attests that archive; the two share no path and each is verified only against its own. Both were extracted and checked file-by-file before deposit, 26300 and 18494 entries, none failing or missing. The 2026-09-03 archive was additionally re-analyzed (114 tracked analysis products byte-identical, 8 identical after two declared normalizations recorded with the deposit, none differing), and *that re-analysis has not been repeated for the extension*.

What is pinned: The Python environment is locked by a resolved dependency lockfile; the Redis image is pinned by digest in the compose file; the matrix records a seed per run and emits its full plan, cost estimate and platform fingerprint *before* launching. Continuous integration provisions Redis from the same compose file the experiments use, runs the full suite, the `WAITAOF` durability suite, a citation-range check over the formal model and the numbers gate described below, and fails if any test is skipped.

What can be reproduced, and at what cost: The matrix is resumable and its cells are independent. The full

plan is 1068 runs at an estimated 25h of wall time on one machine; the cells behind the results in Section 6 are a subset keyed in the tracked per-cell manifest. The raw archive carries the complete per-cell and per-run manifest alongside the runs, so a reader can see what was collected without inferring it from the tables.

Regenerating the paper’s numbers: The tables and every headline scalar in Section 6 are generated by the artifact’s table generator from the tracked inputs its documentation enumerates: per-cell and per-execution metrics, latency under both `fsync` policies, the regime-labeled comparisons, the Redis-kill ablation, coverage, and the write-loss probes. Each emitted value carries a comment naming the filter and arithmetic behind it. The artifact’s build script builds the manuscript and runs that check; it treats an undefined reference and a BibTeX parse error as build failures rather than warnings, because both have produced a clean-looking build of this paper once each. An independent number check re-derives the tables and fails if the manuscript has drifted from the results. It also fails if a generated number is defined and never used, which is the drift a framing revision produces. LaTeX catches a macro used without a definition; it is silent about a claim whose evidence was orphaned when the claim moved. The pooled summary table produced by the analysis tool is by design *not* a source for any number here. All comparisons used by the paper are filtered to an explicit regime; the headline and B3 comparisons use only the crashed regime.

The host-level write-loss probe: Section 6.3.3 needs a block device that stops accepting writes, which needs root, a loop device and a `dm-flakey` target, so the write-loss probe is not part of the test suite. A test that skips wherever those are unavailable is the failure mode this project’s CI gate exists to prevent. What *is* in the suite is everything about the probe that can be checked without a kernel: that its “drop” table really carries `drop_writes`, that its pass-through table carries no feature at all, and that a trial whose acknowledged write vanished is voided rather than counted. The probe itself refuses to report any Redis trial until a self-test has shown, on the bare device, that a file `fsync`-ed before the switch survives and one `fsync`-ed after it does not.

The state-machine figure: Figure 1 is generated by a script that imports the transition set from the implementation. An edge added to or removed from the protocol changes the figure or fails the check; it cannot disagree with the code unnoticed.

Citations: Every bibliography entry was verified to exist before it was written: matched against DBLP’s search API, or resolved through `doi.org`, or (for vendor documentation) fetched and read on a recorded date. The verification script and bibliography metadata are tracked. The raw lookup transcript is not present in the Git repository and is not in the raw-evidence archive either, which carries run data only; it is the one item named in this section for which no external copy yet exists.

9 THREATS TO VALIDITY

9.1 Construct validity

The barrier’s durability benefit is measured on an emulated fault, not a real power cut: Section 6.3.3 injects the fault class `WAITAOF` defends against, loss of the page cache, with a `dm-flakey` target in `drop_writes` mode, and the result is unambiguous (90/90 acknowledged records survive, 90/90 unacknowledged ones do not). What remains is that `drop_writes` is an *emulation*: faithful where it matters (writes after a chosen instant never reach the platter, everything `fsync`-ed before it does) and unfaithful in that real power loss can tear a sector, can lose writes a device claimed to have committed to its own cache, and does not stop at a chosen boundary. Those differences make real hardware worse, so the direction holds and the precision does not: on a device with a lying write cache `fsync` does not mean what the barrier assumes, a property AEP inherits from every system that trusts it. The probe also phase-aligns to a just-completed `fsync`, so 90/90 is the rate for a worst-placed cut rather than a uniform one.

The write-loss result is a refutation, and it is reported as one: Section 6.3.4 runs both arms under the write-loss fault and refutes what we registered: neither arm withheld, because `WAITAOF` returned success after the device had stopped accepting writes. The threat that remains here is the one that result creates rather than settles: the durability guarantee is bounded to storage that reports its own failures, and this paper contains no evidence about storage that does not. The finding, its counts and its bounds are in Section 6.3.4 and are not repeated here.

The write-loss probe runs several layers above hardware, and does not need to be closer: The `dm-flakey` target sits on a loop device backed by a file on the WSL2 root filesystem, itself a virtual disk on the host. The layering is irrelevant to the claim, because `dm-flakey` *is* the boundary under test: the experiment asks whether a `WAITAOF` acknowledgment corresponds to bytes that have crossed a chosen point in the storage path before that point stops accepting writes, and everything below the target is on the far side of it in both arms equally. What the layering *would* threaten is a claim about absolute `fsync` latency on real hardware, which we do not make from this probe, since the barrier’s cost comes from the deployment table in the supplementary material; or a claim that a real power cut cannot do worse, which we explicitly do not make above.

No process-level fault can exercise the barrier’s durability at all: Stated separately because a reader is most likely to have assumed the other way. `appendfsync everysec` defers the `fsync(2)` and not the `write(2)`, so a `SIGKILL` of Redis loses nothing that `appendonly yes` was not already keeping (0/10, and $n=60$ canaries in the ablation cells). **Under a process fault the barrier’s contribution is prevention (Section 6.3.2, $p = 1.9 \times 10^{-6}$, replicated at [6.1, 28.4] effects prevented) and not durability**, and any reading of the barrier as protecting the record from a crashed Redis process is wrong.

The barrier is evaluated under crash faults only: Every prevention measurement here delivers an F3 crash-stop of the state store, a class in which the arms are separated by a *timing* difference and nothing else: AEP-full

has one extra Redis-dependent step, and the fault must land in a window B3 has already left. That window can be narrowed and measured (Section 6.3.2) but not closed: a fault that guarantees `WAITAOF` cannot return also blocks the two Redis calls B3 makes after the same checkpoint. The arm that would separate them by construction, disabling the `fsync` at the checkpoint alone, was designed and not run: it is a configuration change rather than a fault, it instantiates a failure class the model does not contain, and the controlled fault had already reduced the between-session spread to 1 count per class. So the prevention guarantee is evaluated where durability is *raced for* and lost, not where it is *unavailable*; the latter is untested.

The mock provider’s realism: The endpoint is a purpose-built service with a transactionally written ground-truth ledger, run at the delay, timeout and error rates Section 6.1 states. It is realistic in the dimension the experiment turns on (its reconciliation capability, the independent variable of Table 7) and unrealistic in most others: no authentication, no rate limits, no partial writes, no schema evolution, one mutation shape. A real legacy endpoint may also *misdeclare* its capability, which the protocol cannot detect (Section 3.3); we validate the capability’s shape, never its truthfulness.

The dispatch guard assumes trusted same-process code: The guard prevents accidental or ordinary control-flow bypass through the supported dispatch API, and its registry detects copying, reuse and scope mismatch. It is not a cryptographic capability: Python’s underscore naming, object-construction checks and closure placement are not security boundaries, and code running in the same interpreter can reach module internals or bypass the API altogether. The implementation therefore enforces a protocol invariant only under a trusted-code assumption; a hostile-extension threat model would require a separately enforced process or service boundary.

B4 is not Temporal: B4 shares exactly one mechanism with a production durable-execution engine: durable history, replay, and at-least-once activity re-execution. It does not model heartbeating, backoff timing, task queues, versioning, or determinism checking. Its re-execution policy is the vendor’s documented default and is quoted as such [7], but the decision is our code. No claim about Temporal’s throughput, latency, maturity or correctness as a product is made or supported anywhere in this paper. B4 and B4b *bracket* the configuration space rather than survey it: a deployment could set Maximum Attempts to 3, add a heartbeat, or make the activity idempotent. The first two land between our two points on the same duplicate/loss trade, and the third leaves the problem this paper is about.

B4 is our model of a durable-execution engine, and no rate here is the engine’s: Section 6.2 reports the production engine run against the same endpoint in both configurations B4 and B4b instantiate, together with the four things that bound it. The threat this leaves is narrow: every B4 and B4b rate in this paper is our implementation’s, and no rate here should be read as any engine’s. We registered that obligation before collecting the engine data rather than after seeing it. We do not compare the engine’s rates against B4’s, for the reason Section 6.2 gives.

Declared ambiguity is not evaluated as an operational outcome: The paper measures how often AEP declares ambiguity and shows that it never substitutes a silent failure for it. It does *not* show that declaring is better in practice; that would require an operator study, and we have run none. The argument that a declared incident beats an undiscovered one is a normative claim about failure handling, not a result of this evaluation. Relatedly, the artifact has no escalation *mechanism* (Table 4): reaching the terminal state pauses the execution and alerts nobody.

Test counts are not protocol assurance: The suite’s size is evidence about the artifact’s hygiene, not about the protocol’s behavior under faults. The evidence for the latter is the fault-injection matrix.

9.2 Internal validity

The decomposition narrows our own novelty claim: Section 6.3.1 attributes the detection guarantee to the pre-dispatch record and the transition table’s missing re-entry edge, not to the barrier machinery of contribution C2. The two claims therefore rest on different evidence. Detection rests on write-ahead logging plus a state-machine property that is a few lines of Lua, measured across the whole matrix. Prevention rests on the barrier, and is evaluated at one crash point on one host, a narrower scope. Within it the effect replicated across 4 further sessions at [6.1, 28.4] effects prevented, the only session-clustered interval in this paper that excludes zero. The composition being novel was never the argument (Section 7); what is new is the semantics, and the ablation says which half delivers which under the faults it covers.

The cell is no longer one collection under one injector. Section 6.3.2 leads with a *controlled* fault (the container frozen before it is killed, landing in 58.3 ms against 368.4 ms) over 3 sessions and all 3 capability classes. AEP-full’s unwanted-applied rate is 0.0112 against B3’s 0.9778, with a between-session spread of 1 count per class: a measured quantity with a stated spread rather than one draw from a distribution we could only name. **It is bounded by measurement rather than by prediction, and that cost us a pre-registration.** We registered [0.045, 0.081] before collecting, derived from the injector’s landing latency as a fraction of the `WAITAOF` window, and the observed 0.0112 fell *below* it. Both of the model’s premises fail on measurement: 106 of 240 uncontrolled runs acknowledged after that landing had supposedly passed and the same injector call runs 2.8× the bench figure in situ, and the wait is not uniform: 0.0042 of waits fall under 58.3 ms where uniformity assigns 0.0580. Take the measured rate and its spread, not the bound.

The residual is real and irreducible under this fault class. AEP-full still applied 3 effects in 269 executions, and no crash-class fault can drive that to zero, for the reason given under construct validity. The uncontrolled cell agrees in direction and its magnitude still moves: 4 collections ranged 4–20 applied effects out of 30. Attributing that movement to kill latency did not succeed at this design’s precision: +70, -4, +282, +74 ms per session, mean +105 ms, 95% interval [-91, +302], a half-width 1.86 times that mean, containing zero because the sessions disagree. 18 is one draw from that distribution; 0.0112 is not.

The detection finding has no referent outside this artifact: The decomposition above is established by an ablation, AEP-full against itself with the barrier removed, and an ablation is internal by construction. That is the right instrument for *attributing* an effect to a mechanism, but it cannot corroborate the resulting claim from outside. Nothing external to this artifact establishes that a pre-dispatch record without an fsync barrier suffices for detection: the proposition is supported by two systems we wrote, measured by a harness we wrote, against a provider we wrote. The oracle’s independence (below) addresses whether we counted correctly, not whether the finding generalizes beyond our implementation of the pattern. That is the structural cost of the instrument, and it applies to the claim this paper most depends on.

The matrix found defects in our own measurement, including one that would have inflated counts: Three are on the record: a resume defect that merged stale event shards into a re-attempted run’s log, double-counting executions; a supervisor that abandoned its lease instead of waiting, alongside non-idempotent execution-state seeding, both flattering a *baseline* and so biasing against our own hypothesis; and a destructive integration suite run beside a live collection, which replaced the matrix’s Redis underneath it. All three are fixed and the affected runs re-collected; none of the discarded runs contributes a number here. The runner now refuses to start beside a host-level fault injector and records the server’s `run_id` with every run, and the supplementary material gives each incident in full. A fault-injection harness is a stronger verification tool than a unit suite, which cuts both ways: there may be defects it has not found.

Ambiguity is counted from the oracle, not from self-report: A system that declared ambiguity liberally could otherwise score well by saying “I do not know” about everything. Duplicates and lost effects are counted by comparing the provider’s independently written ledger against the system’s records; the analysis is prevented by a parsed source gate from reading live protocol state.

The oracle is independent, not infallible: The provider ledger is written outside the systems under test, and every completed run is reconciled against it, but the provider and ledger share one implementation and one host. One voided run demonstrates that this oracle can fail: its event log and ledger disagreed while sibling and repeated runs agreed. The repository retains the aggregate record of that exclusion, and the voided run’s raw directory (its event log, its ledger and a README giving the reason) is in the published archive under `voided/`, so the disagreement can be read rather than taken on trust. A second provider implementation or an external observer would reduce this common-mode doubt; neither is present.

One author implemented every system under comparison: B0–B2 are small enough to inspect, each carries a machine-readable descriptor that tests check against its implementation, and B4’s re-execution policy is defended against the vendor’s own documentation in the artifact. That is mitigation, not elimination: a baseline written by the people proposing the alternative is a structural conflict, and the artifact is released partly so the baselines can be re-implemented by someone else.

9.3 External validity

Single node, single trust domain: One Redis instance, no replica, no consensus. Table 4 lists what this forecloses. One residual interacts with the results: an AOF rewind can un-fence a lease, since lock keys are by design not barriered, and the fix is HA rather than anything local.

Concurrency coverage is narrow: The harness exercises crash and takeover with two workers, so stale-owner and successor paths are covered, but it does not sustain live contention among many workers for the same execution or endpoint resource. The evaluation therefore supports fencing and recovery under the measured takeover schedule, not a scalability or fairness claim for high-contention deployments. Concurrent-claim rejection is exercised by the test suite; the fault-injection matrix supplies no contention-regime evidence for P1.

Platform, and a development/measurement split: All measurements were collected on Ubuntu inside WSL2 on Windows 11, with Docker Desktop port forwarding in the path, on mains power with sleep and hibernation disabled. Development and part of the test suite run on the Windows host; *every* number in Section 6 comes from the Linux tree against real Redis. The locally runnable Windows suite and the artifact’s Linux suite are checked separately, while the Linux evidence is the one the measurements rest on. Absolute latency and throughput are platform-bound; only comparisons *between* systems sharing this platform are meaningful, which is why Table 9 is presented as increments over a common floor.

We treat this as a limitation rather than a defect to be fixed before publication, and the specific thing it costs is worth naming. Every absolute median here carries an unmeasured platform term, and the way to measure it is not to re-collect the evaluation on bare metal but to re-collect *one frozen cell* there and compare: the crash-free *everysec* cell of AEP-full and B3 is thirty runs and about an hour, it is the cell both barrier figures come from, and its archive manifest makes the comparison exact rather than approximate. Until that exists, a reader should treat the increments as this platform’s and the ordering as the claim.

The motivating traces come from our own harness: Section 2 reproduces failures in baselines we implemented, not in deployed systems. It establishes that the failure modes are reachable under the stated fault model; it is not evidence about their frequency in production deployments, and we have none. A field study of how often such systems meet non-idempotent endpoints without idempotency keys would strengthen the motivation considerably.

Coverage of the matrix is partial, and the gaps are named: The full plan is 1068 runs; we collected 432, comprising 3780 executions over 126 cells, chosen to answer the research questions rather than to fill the grid. The supplementary material enumerates six named gaps: five since closed and recorded there with what closing them changed, and one still open, a run that did not complete. Two qualifications from that enumeration bear on the main results and are stated here rather than only there: every collection in this paper comes from *one host*, so the prevention effect’s dependence on host timing is measured *on* that host rather than established *across* hosts; and completing

a previously partial cell changed its value materially, so a partial cell should be read as unrepresentative rather than merely imprecise. A rate we did not measure is absent from the tables rather than interpolated, and the manifest shipped with the results states the run count for every cell.

One endpoint for the cost measurement: RQ3's cells were collected against a single endpoint. Overhead should not depend on an endpoint's reconciliation capability in a crash-free run, since no read-back occurs, but that is an argument and not a measurement.

9.4 Conclusion validity

What fifteen runs per arm settled, and what they did not: The timing cells were re-collected at fifteen crash-free runs per arm under both `fsync` policies, and the two questions they were meant to separate came apart. The barrier's cost under `everysec` is now pinned: 1939.7 ms, [1855.8, 1962.4]. The protocol's cost apart from the barrier is not, under either pre-registered reading of B3's bimodal arm, and both intervals contain zero (Section 6.4). That is a statement about resolution and not about the effect: a residual of a few tens of milliseconds on a two-second call is below what fifteen runs of this workload can separate from nothing, and more runs would narrow it rather than move it. A deployment that needs a latency budget should measure its own. The four medians in the supplementary's deployment table are the earlier three-run cells and are shown as the deployment points they were collected to compare, not as the source of the costs quoted above.

A failure to reject is not a null result here, and under the sign test that is a theorem: Two comparisons in this paper fail to exclude zero for reasons of precision rather than of effect: the kill-latency attribution above, whose half-width is wider than the mean it brackets, and the capability-class sweep reported in the supplementary material, where the applied-effect column moved, contradicting our own registered prediction, while the session-clustered interval still contains zero because the sessions disagree with one another.

For both, the weaker reading is not available. Each rests on four paired sessions, and with n paired sessions the smallest attainable two-sided sign-test p -value is $2/2^n$, which at $n = 4$ is 0.125. **Under the sign test, no effect size however large could have produced a rejection at $\alpha = 0.05$ from these designs.** Six sessions is the smallest at which that test becomes capable of rejecting at all ($2/2^6 = 0.03125$).

That is a statement about the sign test and not about four-session designs. The same design rejects under the t -analysis we report elsewhere: the prevention replication is four paired sessions and its interval excludes zero (Section 6.3.2). So the asymmetry a reader may notice is real, and the reason is the quantity rather than the arithmetic. We use the t -interval wherever the session differences are the measurement, which is all three of these comparisons, and we quote the sign-test floor only to say what a rank test could have detected at this n . Read that way, all three are reported on one standard: an interval, and its width beside its mean. Neither of the two should be read as evidence that the effect is absent, and neither is offered as one.

We stopped at four sessions by choice rather than for want of machine time, and the class sweep is reported as a

bound rather than as a test. A further session was collected far enough to establish that it could not be pooled with the first four: the original sessions run one arm to completion and then the other, while the harness now alternates the arms run by run, a change made because the block order makes arm and drift collinear and therefore not separable by any number of sessions. Adding interleaved sessions to blocked ones would put two designs inside one paired test. Each session directory carries a note recording its order and what that confounds.

One results directory was destroyed during tooling work and recovered from the archive: The raw runs behind the `appendfsync always` cell were deleted from a working checkout by a collection script whose clean path defaulted to on, and were restored from the verified evidence archive, whose manifest covers them by name and digest; the derived results are unchanged and remain recomputable from raw.

Timing hygiene, and what it cost us: The host's modern-standby setting suspended it mid-run during earlier collection, silently inflating wall-clock durations. We now require both a pre-run declaration that the host cannot suspend and a per-run wall-versus-monotonic check. The gate is strict enough that *no* run collected before it existed contributes a timing number, and it costs us coverage: recovery-latency distributions are not reported (the supplementary material, *RQ4: recovery*) because the gated crashed runs are too few, and we prefer the gap to ungated numbers.

9.5 Where the protocol should not be used

If the endpoint accepts an idempotency key, or can be enrolled in a transaction, or the effect can be made idempotent, then a durable-execution engine gives a strictly stronger guarantee at lower cost and AEP is the wrong tool. Its cost is real: two `fsync` barriers, which under `appendfsync everysec` dominate the protocol's latency. We do not put a factor on that dominance: the denominator is the protocol's cost apart from the barrier, and at fifteen runs per arm that quantity is [-122.6, 120.1] ms, an interval containing zero (Section 6.4). A ratio whose denominator is not distinguishable from zero is not a measurement. AEP is for the case where the precondition those systems require cannot be obtained, and its value is a declared residual rather than a better guarantee.

REFERENCES

- [1] P. Helland, "Life beyond distributed transactions: An apostate's opinion," in *Proceedings of the 3rd Biennial Conference on Innovative Data Systems Research (CIDR)*, 2007, pp. 132–141.
- [2] S. Burckhardt, C. Gillum, D. Justo, K. Kallas, C. McMahon, and C. S. Meiklejohn, "Durable functions: Semantics for stateful serverless," *Proceedings of the ACM on Programming Languages*, vol. 5, no. OOPSLA, pp. 1–27, 2021.
- [3] S. Zhuang, S. Wang, E. Liang, Y. Cheng, and I. Stoica, "ExoFlow: A universal workflow system for exactly-once DAGs," in *Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2023, pp. 269–286.
- [4] Temporal Technologies, "Activity definition," <https://docs.temporal.io/activity-definition>, 2026, accessed 2026-08-06.
- [5] M. J. Fischer, N. A. Lynch, and M. S. Paterson, "Impossibility of distributed consensus with one faulty process," *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985.

- [6] T. D. Chandra and S. Toueg, "Unreliable failure detectors for reliable distributed systems," *Journal of the ACM*, vol. 43, no. 2, pp. 225–267, 1996.
- [7] Temporal Technologies, "Retry policies," <https://docs.temporal.io/encyclopedia/retry-policies>, 2026, accessed 2026-08-06.
- [8] Redis, "Redis persistence," https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/, 2026, accessed 2026-08-07.
- [9] C. G. Gray and D. R. Cheriton, "Leases: An efficient fault-tolerant mechanism for distributed file cache consistency," in *Proceedings of the 12th ACM Symposium on Operating Systems Principles (SOSP)*, 1989, pp. 202–210.
- [10] Redis, "WAITAOF," <https://redis.io/docs/latest/commands/waitaof/>, 2026, accessed 2026-08-07.
- [11] M. Kleppmann, "How to do distributed locking," <https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html>, 2016, accessed 2026-08-07.
- [12] M. Burrows, "The Chubby lock service for loosely-coupled distributed systems," in *Proceedings of the 7th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2006, pp. 335–350.
- [13] P. Hunt, M. Konar, F. P. Junqueira, and B. Reed, "ZooKeeper: Wait-free coordination for internet-scale systems," in *2010 USENIX Annual Technical Conference (USENIX ATC)*, 2010. [Online]. Available: <https://www.usenix.org/conference/usenix-atc-10/zookeeper-wait-free-coordination-internet-scale-systems>
- [14] Redis, "Distributed locks with Redis," <https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/>, 2026, accessed 2026-08-10.
- [15] H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Transactions on Database Systems*, vol. 6, no. 2, pp. 213–226, 1981.
- [16] C. Mohan, D. Haderle, B. G. Lindsay, H. Pirahesh, and P. M. Schwarz, "ARIES: A transaction recovery method supporting fine-granularity locking and partial rollbacks using write-ahead logging," *ACM Transactions on Database Systems*, vol. 17, no. 1, pp. 94–162, 1992.
- [17] C. Richardson, "Pattern: Transactional outbox," [Microservices.io](https://microservices.io/patterns/data/transactional-outbox.html), accessed 2026-08-11. [Online]. Available: <https://microservices.io/patterns/data/transactional-outbox.html>
- [18] J. Gray, "The transaction concept: Virtues and limitations," in *Proceedings of the 7th International Conference on Very Large Data Bases (VLDB)*, 1981, pp. 144–154. [Online]. Available: <https://dl.acm.org/doi/10.5555/1286831.1286846>
- [19] H. F. Korth, E. Levy, and A. Silberschatz, "A formal approach to recovery by compensating transactions," in *Proceedings of the 16th International Conference on Very Large Data Bases (VLDB)*, 1990, pp. 95–106. [Online]. Available: <https://www.vldb.org/conf/1990/P095.PDF>
- [20] H. Garcia-Molina and K. Salem, "Sagas," in *Proceedings of the 1987 ACM SIGMOD International Conference on Management of Data*, 1987, pp. 249–259.
- [21] P. Helland, "Idempotence is not a medical condition," *Communications of the ACM*, vol. 55, no. 5, pp. 56–65, 2012.
- [22] J. Jena and S. Dalal, "The Idempotency-Key HTTP header field," Internet Engineering Task Force, Internet-Draft draft-ietf-httpapi-idempotency-key-header-07, Oct. 2025, work in progress; expired 2026-04-18. [Online]. Available: <https://datatracker.ietf.org/doc/html/draft-ietf-httpapi-idempotency-key-header-07>
- [23] Stripe, "Idempotent requests — Stripe API reference," https://docs.stripe.com/api/idempotent_requests, 2026, accessed 2026-09-08.
- [24] Adyen, "API idempotency," <https://docs.adyen.com/development-resources/api-idempotency/>, 2026, accessed 2026-09-08.
- [25] C. Mohan, B. Lindsay, and R. Obermarck, "Transaction management in the R* distributed database management system," *ACM Transactions on Database Systems*, vol. 11, no. 4, pp. 378–396, 1986.
- [26] D. Skeen, "Nonblocking commit protocols," in *Proceedings of the 1981 ACM SIGMOD International Conference on Management of Data*, 1981, pp. 133–142.
- [27] J. C. Corbett, J. Dean, M. Epstein, A. Fikes, C. Frost, J. J. Furman, S. Ghemawat, A. Gubarev, C. Heiser, P. Hochschild, W. Hsieh, S. Kanthak, E. Kogan, H. Li, A. Lloyd, S. Melnik, D. Mwaura, D. Nagle, S. Quinlan, R. Rao, L. Rolig, Y. Saito, M. Szmaniak, C. Taylor, R. Wang, and D. Woodford, "Spanner: Google's globally-distributed database," in *10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2012. [Online]. Available: <https://www.usenix.org/conference/osdi12/technical-sessions/presentation/corbett>
- [28] S. T. V. Setty, C. Su, J. R. Lorch, L. Zhou, H. Chen, P. Patel, and J. Ren, "Realizing the Fault-Tolerance promise of cloud storage using locks with intent," in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2016, pp. 501–516. [Online]. Available: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/setty>
- [29] D. Georgakopoulos, M. Hornick, and A. Sheth, "An overview of workflow management: From process modeling to workflow automation infrastructure," *Distributed and Parallel Databases*, vol. 3, no. 2, pp. 119–153, 1995.
- [30] Uber Technologies, "Cadence documentation: Workflows and activities," <https://cadenceworkflow.io/docs/concepts/workflows>, 2026, accessed 2026-09-08.
- [31] ReState, "Key concepts: Durable execution," <https://docs.restate.dev/foundations/key-concepts>, 2026, accessed 2026-09-08.
- [32] Amazon Web Services, "Best practices for Step Functions," <https://docs.aws.amazon.com/step-functions/latest/dg/sfn-best-practices.html>, 2026, accessed 2026-09-08.
- [33] —, "Idempotency and retries — AWS durable execution SDK," <https://docs.aws.amazon.com/durable-execution/patterns/best-practices/idempotency/>, 2026, accessed 2026-09-08.
- [34] A. Skiadopoulos, Q. Li, P. Kraft, K. Kaffes, D. Hong, S. Mathew, D. Bestor, M. Cafarella, V. Gadeppally, G. Graefe, J. Kepner, C. Kozyrakis, T. Kraska, M. Stonebraker, L. Suresh, and M. Zaharia, "DBOS: A DBMS-oriented operating system," *Proceedings of the VLDB Endowment*, vol. 15, no. 1, pp. 21–30, 2021.
- [35] H. Zhang, A. Cardoza, P. B. Chen, S. Angel, and V. Liu, "Fault-tolerant and transactional stateful serverless workflows," in *Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2020, pp. 1187–1204.
- [36] Z. Jia and E. Witchel, "Boki: Stateful serverless computing with shared logs," in *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*, 2021, pp. 691–707.
- [37] Temporal Technologies, "Detecting activity failures," <https://docs.temporal.io/encyclopedia/detecting-activity-failures>, 2026, accessed 2026-08-06.
- [38] I. K. Mansoor, A. Phadke, and P. Rana, "Verified tool calls improve LLM agent reliability under non-atomic failures," *CoRR*, vol. abs/2608.02645, 2026.
- [39] M. Balakrishnan, A. Bharambe, D. Testuggine, D. Geraghty, D. Mao, V. Venkat, I. Mironov, R. Baradi, G. Aiyyer, and V. Dudin, "LogAct: Enabling agentic reliability via shared logs," *CoRR*, vol. abs/2604.07988, 2026.
- [40] J. He and D. Yu, "Sovereign execution broker: Enforcing certificate-bound authority in agentic control planes," *CoRR*, vol. abs/2606.20520, 2026.
- [41] Y. Zheng, Y. Yang, W. Zhang, and A. Quinn, "ACRFence: Preventing semantic rollback attacks in agent checkpoint-restore," *CoRR*, vol. abs/2603.20625, 2026.
- [42] A. Rebello, Y. Patel, R. Alagappan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Can applications recover from fsync failures?" in *2020 USENIX Annual Technical Conference (USENIX ATC)*, 2020, pp. 753–767. [Online]. Available: <https://www.usenix.org/conference/atc20/presentation/rebello>
- [43] J. Corbet, "PostgreSQL's fsync() surprise," <https://lwn.net/Articles/752063/>, 2018, lwn.net. Accessed 2026-09-08.
- [44] A. Ganesan, R. Alagappan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Redundancy does not imply fault tolerance: Analysis of distributed storage reactions to single errors and corruptions," in *15th USENIX Conference on File and Storage Technologies (FAST)*, 2017, pp. 149–166. [Online]. Available: <https://www.usenix.org/conference/fast17/technical-sessions/presentation/ganesan>
- [45] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC)*, 2014, pp. 305–319.
- [46] R. Alagappan, A. Ganesan, E. Lee, A. Albarghouthi, V. Chidambaram, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Protocol-aware recovery for consensus-based storage," in *16th USENIX Conference on File and Storage Technologies (FAST)*, 2018, pp. 15–32. [Online]. Available: <https://www.usenix.org/conference/fast18/presentation/alagappan>

- [47] K. M. Chandy and L. Lamport, "Distributed snapshots: Determining global states of distributed systems," *ACM Transactions on Computer Systems*, vol. 3, no. 1, pp. 63–75, 1985.
- [48] P. Carbone, G. Fóra, S. Ewen, S. Haridi, and K. Tzoumas, "Lightweight asynchronous snapshots for distributed dataflows," *CoRR*, vol. abs/1506.08603, 2015. [Online]. Available: <https://arxiv.org/abs/1506.08603>
- [49] J. Y. Halpern and Y. Moses, "Knowledge and common knowledge in a distributed environment," *Journal of the ACM*, vol. 37, no. 3, pp. 549–587, 1990.
- [50] G. Wang, L. Chen, A. Dikshit, J. Gustafson, B. Chen, M. J. Sax, J. Roesler, S. Blee-Goldman, B. Cadonna, A. Mehta, V. Madan, and J. Rao, "Consistency and completeness: Rethinking distributed stream processing in Apache Kafka," in *Proceedings of the 2021 International Conference on Management of Data (SIGMOD)*, 2021, pp. 2602–2613.
- [51] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Transactions on Computer Systems*, vol. 2, no. 4, pp. 277–288, 1984.
- [52] T. S. Pillai, V. Chidambaram, R. Alagappan, S. Al-Kiswani, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "All file systems are not created equal: On the complexity of crafting crash-consistent applications," in *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2014, pp. 433–448. [Online]. Available: <https://www.usenix.org/conference/osdi14/technical-sessions/presentation/pillai>
- [53] M. Zheng, J. A. Tucek, D. Huang, F. Qin, M. Lillibridge, E. S. Yang, B. W. Zhao, and S. Singh, "Torturing databases for fun and profit," in *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2014, pp. 449–464. [Online]. Available: https://www.usenix.org/conference/osdi14/technical-sessions/presentation/zheng_mai
- [54] J. Mohan, A. Martinez, S. Ponnappalli, P. Raju, and V. Chidambaram, "Finding crash-consistency bugs with bounded black-box crash testing," in *Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association, 2018, pp. 33–50. [Online]. Available: <https://www.usenix.org/conference/osdi18/presentation/mohan>
- [55] J. Bornholt, A. Kaufmann, J. Li, A. Krishnamurthy, E. Torlak, and X. Wang, "Specifying and checking file system crash-consistency models," in *Proceedings of the 21st International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2016, pp. 83–98.
- [56] H. S. Gunawi, T. Do, P. Joshi, P. Alvaro, J. M. Hellerstein, A. C. Arpaci-Dusseau, R. H. Arpaci-Dusseau, K. Sen, and D. Borthakur, "FATE and DESTINI: A framework for cloud recovery testing," in *8th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2011. [Online]. Available: <https://www.usenix.org/conference/nsdi11/fate-and-destini-framework-cloud-recovery-testing>
- [57] P. Alvaro, J. Rosen, and J. M. Hellerstein, "Lineage-driven fault injection," in *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*, 2015, pp. 331–346.
- [58] K. Kingsbury and P. Alvaro, "Elle: Inferring isolation anomalies from experimental observations," *Proceedings of the VLDB Endowment*, vol. 14, no. 3, pp. 268–280, 2020.